



A Knowledge Base with which You Can Parameterize Robot Plans for Multiple Mixing Actions

Master Thesis

Naser Azizi, Sorin Arion

Prüfer der Master Thesis:

1. Prof. Michael Beetz PhD

2. Prof. Dr. Nico Hochgeschwender

Supervisor

Dr. Michaela Kümpel, Vanessa Hassouna

Eidesstattliche Erklärung

Hiermit erklären wir, dass die vorliegende Arbeit selbstständig angefertigt, nicht anderweitig zu Prüfungszwecken vorgelegt und keine anderen als die angegebenen Hilfsmittel verwendet habe. Sämtliche wissentlich verwendete Textausschnitte, Zitate oder Inhalte anderer Verfasser wurden ausdrücklich als solche gekennzeichnet.

Bremen, den September 13, 2024

Naser Azizi, Sorin Arion

Motivation

Although industrial robots have been considered standard for many years, the widespread adoption of AI-based autonomous household robots is still far from being anticipated in every household. Obstacles such as unfamiliar surroundings and a limited knowledge base can significantly hinder the ability of an autonomous robot to effectively respond in a given situation. Hence, the robot must precisely plan for every minor detail, even those that might appear trivial to us humans.

To achieve the defined objectives, a proficient and comprehensive robotic system including control, perception, navigation, and knowledge is necessary. An illustrative goal could be cooking. To execute this task, the robot must possess various capabilities, including placing items, grasping objects, pouring liquids, cutting ingredients, and mixing components. These tasks seem trivial to us humans, but for robots, they require extensive implementation and information.

In this thesis, we aim to introduce an approach detailing how a robotic system can execute mixing actions effectively. The challenges arise from the diverse methods of mixing, which further vary based on the specific ingredients being combined. Additionally, the consideration of compatible mixing tools and mixing containers is crucial, as not every tool can be utilized with every container.

Through the incorporation of a knowledge graph containing rules related to various actions, we aim to empower the robotic system to make informed decisions on the appropriate motions to employ. This decision-making process will take into account the specific task at hand and the involved ingredients. By incorporating this knowledge graph, we aim to advance towards autonomous robots capable of engaging in cooking activities.

Contents

1	Introduction	5
2	Related Work	7
3	Introduction to Hardware and Software components	10
3.1	Robotic section: ROS and PyCRAM	10
3.1.1	ROS	10
3.1.2	Pr2	11
3.1.3	PyCram	11
3.1.4	PyBullet	12
3.2	Knowledge Section: Ontologies and Rules	13
3.2.1	Ontology	13
3.2.2	SWRL	14
3.3	Software Libraries	15
3.3.1	RDFLib	15
3.3.2	Flask	16
3.3.3	Owlready	16
3.3.4	WikiHow-Instruction-Extraction	17
4	Data Acquisition	18
4.1	Task variations	19
4.2	Task Analysis and Defintion	21
4.3	Data Acquisition Conclusion	30
5	Data Representation	31
5.1	The Knowledge Base	32
5.1.1	Ingredients	33
5.1.2	Tools and containers	34
5.1.3	Tasks	35
5.1.4	Motions	36
5.2	Rules	37
5.2.1	Mixing	38
5.2.2	Stirring	39
5.2.3	Beating	39
5.2.4	Whisking	40
5.2.5	Folding	40

6	Execution on the Robot	42
6.1	Simulation Environment	42
6.2	Control Primitives	45
6.3	Implementation in PyCram	46
6.3.1	Robots Plan Execution	46
6.3.2	Asserted Knowledge	47
6.3.3	Inferring Knowledge	47
6.3.4	MixingActionSWRL	47
6.3.5	MixingActionDesignator	48
6.3.6	Implementation	48
6.3.7	Simulated Motions	50
6.4	Simulation to Real-World Gap	52
6.5	Challenges and Difficulties, Summary of the Simulation	53
6.6	Evaluation	54
7	Knowledge Graph Visualization Tool	60
7.1	Main concept	60
7.2	Features	61
7.2.1	Graph Visualization	61
7.2.2	Graph Processing	62
7.2.3	Load Ontology	65
7.2.4	Search Classes	66
7.2.5	Custom Inference - Mixing	67
7.2.6	Query Builder	68
7.3	Limitations	71
8	Conclusion	72
8.1	Summary	72
8.2	Future Work	73
9	Appendix	78
9.1	Video Analysis	78
9.2	Evaluation	80
9.2.1	Mixing Task	81
9.2.2	Stirring Task	82
9.2.3	Beating Task	83
9.2.4	Whisking Task	85
9.2.5	Folding Task	86

1 Introduction

This master’s thesis addresses the issue of mixing and its variations in the context of cooking as high-level tasks, taking into account contextual knowledge, to derive statements about which mixing movements a robotic agent should perform using a knowledge base.

Before presenting the structure of the master’s thesis, we address five essential aspects to provide readers with a better overview. The problem at hand involves various tasks in the world of cooking and baking. While humans learn over their lifetime how to prepare recipes and intuitively execute the necessary actions, a robot must have these required actions readily available. A robot’s tasks in cooking can be diverse, such as cutting, pouring, or mixing. This thesis focuses on the task of mixing and examines the issue of different mixing actions and the context in which they are performed.

The importance of considering various movements to achieve desired results in mixing is emphasized, as a single movement cannot suffice for different outcomes. Different movements must be taken into account to achieve the desired results of a mixing action. The difficulty arises from the lack of a systematic approach to determine which mixing movement should be performed under specific circumstances. A systematic approach, primarily utilizing video material, is necessary to identify the relationships between the actions performed by humans, the ingredients used, and the associated movements. The biggest challenge lies in deciding how to model this knowledge in a symbolic knowledge base so that the relevant knowledge is usable for the robot.

To solve this problem, the analysis of videos is employed to determine how general principles can be inferred from individual cases, especially regarding human actions during the mixing of ingredients. Both the nature of the ingredients and the movements executed are considered. The identified relationships are modeled in a knowledge base, allowing a robotic agent to derive possible movements with the necessary parameters for execution. The knowledge base is symbolically modeled, enabling theoretical support for execution by many robotic agents.

The knowledge base should be used by to execute various motions accordingly, taking into account defined parameters that adapt to the geometric characteristics of the containers and tools. This involves extracting geometric information and defining specific action ranges for the motions.

This work contributes to research by enabling robotic agents to query the required knowledge for mixing actions. The implementation of our knowledge base aims at developing

robots capable of performing non-trivial tasks autonomously in household settings without a predefined environment.

Our work is divided into multiple chapters that will cover every aspect of our research and practical work. First, we intend to analyze existing research, particularly examining studies that delve into diverse mixing motions and exploring related systems that contain a queryable knowledge graph(2).

In the following section, we will introduce the reader with the frameworks employed to accomplish our objectives (3). Subsequently, we will present our approach to data acquisition and data representation (4, 5). In this chapters, we aim to explain the mixing tasks under consideration and articulate our methodology for representing this data to enable effective querying. In the subsequent section, we will illustrate the implementation of our rules, with the ultimate aim of determining the appropriate motion to be employed.

To validate our concept, we have opted to simulate various scenarios and assess their outcomes, demonstrating the efficacy of our implemented system (6).

Before we will delve into the discussion of our results and draw conclusions to wrap up our work, we want to present a newly implemented web framework for graph visualization (7).

The following graphic is intended to provide an overview and will be referenced in each chapter to indicate which part of the work the chapter is addressing.

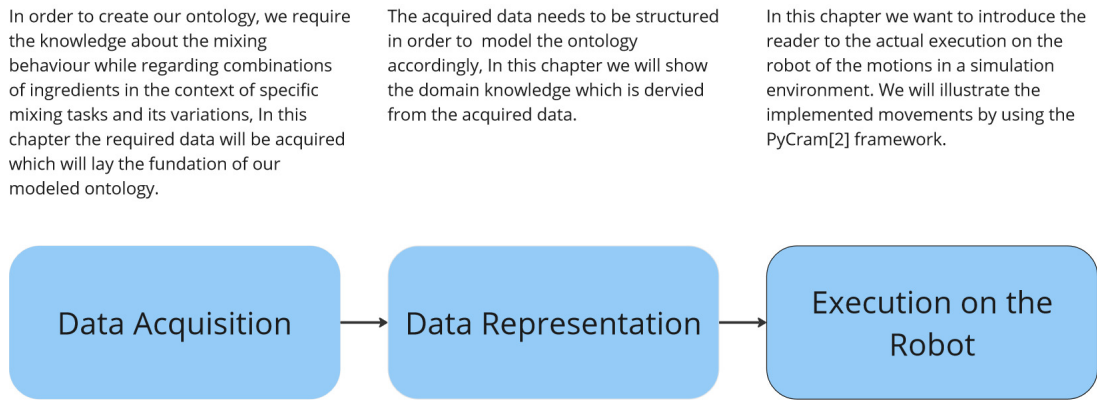


Figure 1.1: Structure overview

2 Related Work

There are some approaches for mixing tasks in the kitchen domain. From high level symbolic to low level motions and learning approaches, these different approaches, tries to tackle the challenge of solving mixing in the kitchen environment.

The concepts presented in the paper *Towards a Knowledge Engineering Methodology for Flexible Robot Manipulation in Everyday Tasks* [22] aims to equip robotic agents with necessary information about how to execute everyday tasks (with focus on cutting tasks) in unknown environments for the household domain. From high level plans the presented concept breaks down the cutting task, part of some robot plan, into executable motions regarding cutting fruits and vegetables. These motions are parameterized by some technique and repetitions to achieve the agents objective. The presented concept does not require fully available knowledge about the agents environment, instead the robot should be capable of recognizing certain objects for cutting operations. In designing our knowledge representation, we follow the presented results from the paper *Towards a Knowledge Engineering Methodology for Flexible Robot Manipulation in Everyday Tasks* [22] and attempt to apply the concept to the domain of mixing.

Another work that served as a foundation for the general design of our knowledge base was the doctoral thesis *Actionable Knowledge Graphs - How Daily Activity Applications Can Benefit from Embodied Web Knowledge* [21]. From this work, we extracted valuable information not only for the creation of our ontology but also for the process of data acquisition. Additionally, the papers *Task Learning Using Actionable Knowledge Graphs* [23] and *Translating Actionable Knowledge Graphs into Robot Action Execution* [6] introduce further concepts for designing a knowledge representation for the execution of robotic plans.

When considering the use case from the perspective of manipulative execution, this work follows the concept presented in the paper *Steps Towards Generalized Manipulation Action Plans - Tackling Mixing Task* [17] and in the thesis *Exploring the Use of Designators for Improved Contextualization in Robotic Mixing Tasks* [34]. Various manipulative tasks from the mixing domain are introduced there, which we use as a foundation and build upon with adapted motions. The architecture used in that work, *PyCram* [2], is also utilized in our work for the planning and execution of motions. The motions presented in the mentioned works served as the basis for the motions we used in our plans.

BakeBot [11] realised on the *Pr2* robotics platform attempts to achieve baking cookies. An implementation of locating relevant things for mixing tasks like a bowl and ingredients has

been realised for semi-structured environments. An algorithm to perform mixing motions has been implemented as well, to mix ingredients with different characteristics into an uniform dough. These motions are limited to a simple circular and linear mixing motion. The authors follow an bottom-up approach which is inherently motion driven rather than a symbolic one which attempts to break down high level tasks into executable low level motions.

Robotic roommates making pancakes [7] is another paper which shows how robot behave in a baking environment. This article is about two robots that bake pancakes through collaboration. Although the mixing motions are not explicitly mentioned in this article, it shows an approach to actions through reasoning. This approach is similar to our work. As a result, the agent is able to reason a sequence of actions based on an input.

Some interesting approaches pursue robot learning by tracking human movement [13]. By learning these movements, they can be transferred to primitive motions and parameterized based on tracking. In our work, we do not pursue learning through demonstrations; however, this paper mentions parameterizing motion primitives (such as a radius), which is similarly addressed in our work.

FluidLab [38] is a simulation environment for different kinds of manipulation tasks regarding liquids. Its underlying engine uses differentiable physics enabling reinforcement learning and optimization techniques in manipulation to utilize the engine, to achieve several tasks including liquids and solids, like mixing tasks.

Our mixing approach will be most similar to the *FoodCutting* approach, in which we model symbolic knowledge about how to perform mixing tasks, which technique should be used and inferring parameters for the execution of the underlying motion.

As we also developed a graph visualization framework, we have to consider existing frameworks that work similarly.

Stardog Explorer [33] is a tool designed to help users visualize and query data in the *Stardog* graph database. It has a user-friendly interface, a drag-and-drop *SPARQL* query builder, and several options for visualizing data, like graphs, charts, and tables. These features make it easier for users to understand complex data relationships without needing advanced technical skills.

Neo4j Bloom [25] is a visualization and exploration tool designed for the *Neo4j* graph database. It provides an intuitive interface for users to query and explore their data visually. *Neo4j Bloom* [25] supports natural language search, allowing users to construct queries in plain English, which are then translated into *Cypher*, *Neo4j's* query language.

Gephi [5] is an open-source network analysis and visualization software that is widely used for exploring and understanding complex networks. It offers dynamic filtering, real-time network manipulation, and a variety of layout algorithms to help users visualize their data in meaningful ways. *Gephi* [5] is particularly popular in research and academic settings

due to its flexibility and extensibility.

The tool developed in this thesis shares some features with the mentioned frameworks, such as visualizing class hierarchies and query building. However, it is specifically tailored to our use case. It provides customized visualization and querying capabilities that better meet the unique needs of our ontologies, while also including a custom inference builder, making data analysis more efficient and effective for our particular application.

3 Introduction to Hardware and Software components

This chapter serves as an introduction to the frameworks, software, and libraries we use. The work we present can be simplified explained as a knowledge base containing parameters that a robot can use to perform specific actions.

Firstly, we would like to introduce the robotic component, which acts as the interface between the knowledge base we have created and the robot, followed by an introduction to ontologies and knowledge bases.

3.1 Robotic section: ROS and PyCRAM

The main framework we use is *PyCRAM* [2], which serves as an interface for various software components such as knowledge, perception, or manipulation. This framework utilizes another framework, *ROS* (Robotic Operating System) [32], to communicate with the different robot components. These two frameworks are now introduced in the following.

3.1.1 ROS

ROS [32], which stands for Robot Operating System, is an open-source middleware framework designed to develop and control robots. Despite its name, *ROS* [32] is not a traditional operating system but rather a set of software libraries and tools that help in building and managing robot software. It provides a standardized and modular approach to developing robotic systems, allowing for easier collaboration and code reuse in the robotics community. Key features and components of *ROS* [32] include:



Figure 3.1: ROS [32]

- **Nodes:** *ROS* [32] systems are organized into nodes, which are individual processes that perform specific tasks. Nodes communicate with each other by passing messages over topics, creating a decentralized and modular architecture.
- **Topics:** Nodes exchange data through topics, which are named buses over which messages are passed. This publish/subscribe communication model allows for asynchronous and loosely coupled interactions between nodes.
- **Launch files:** *ROS* [32] uses launch files to specify how to start multiple nodes and configure the system. This helps simplify the setup of complex robotic systems.
- **Master:** The ROS Master is responsible for managing the communication between nodes by keeping track of publishers, subscribers, and services. It facilitates the discovery and connection of nodes within a *ROS* [32] network.

3.1.2 Pr2

Introduced in 2010 by Willow Garage, the *PR2* [1] stands as an advanced research robot. Boasting multiple joints and 20 degrees of freedom, this robot excels in autonomous navigation and the manipulation of a diverse array of objects, making it an ideal choice for our specific needs. Additionally, it is equipped with a *HeadStereoCamera* that can be used to perceive the surroundings.

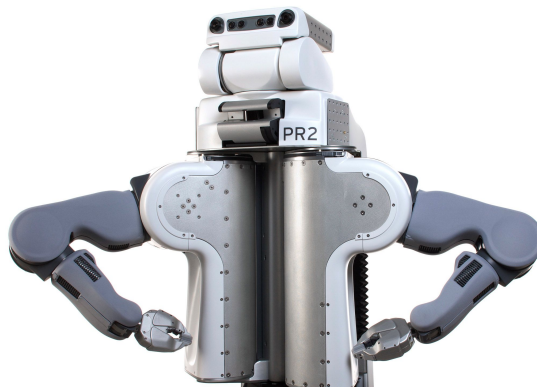


Figure 3.2: PR2 [1]

3.1.3 PyCram

PyCRAM [2] is a toolbox for designing, implementing and deploying software on autonomous robots. The framework provides various tools and libraries for aiding in robot software development as well as geometric reasoning and fast simulation mechanisms to

develop cognition-enabled control programs that achieve high levels of robot autonomy. *PyCRAM* [2] is developed in the *Python*-programming language with support for the *ROS* [32] middleware which is used for communication with different software components as well as the robot.

CRAM [8] (Cognitive Robot Abstract Machine) is a software toolbox for the design, the implementation, and the deployment of cognition-enabled autonomous robots performing everyday manipulation activities. *CRAM* [8] equips autonomous robots with lightweight reasoning mechanisms that can infer control decisions rather than requiring the decisions to be pre-programmed. This way *CRAM*-programmed autonomous robots are much more flexible, reliable, and general than control programs that lack such cognitive capabilities. *CRAM* [8] does not require the whole domain to be stated explicitly in an abstract knowledge base. Rather, it grounds symbolic expressions in the knowledge representation into the perception and actuation routines and into the essential data structures of the control programs.

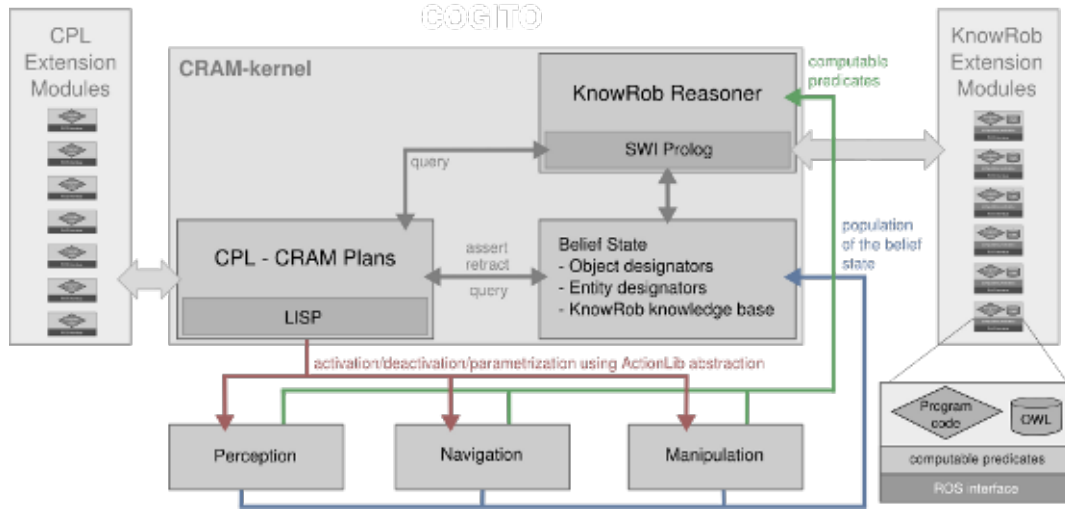


Figure 3.3: *CRAM* [8] Language Architecture

3.1.4 PyBullet

PyBullet[12] is an open-source physics engine and 3D simulation library used for robotics, machine learning, and computer graphics research. It offers accurate physics simulation, 3D rendering, robotics support, and seamless integration with machine learning frameworks. With a *Python*-API and cross-platform compatibility, it's a versatile tool for simulating complex environments and interactions.

3.2 Knowledge Section: Ontologies and Rules

The parameters inferred for various robot actions come from a knowledge base. In the following, the principle of an ontology, as well as the concept of rules, which play a crucial role in parameter inference, will be introduced.

3.2.1 Ontology

Ontologies [27] are structured frameworks that provide a formal representation of knowledge within a specific domain. They play a crucial role in knowledge representation, facilitating the organization and sharing of information in a way that is both machine-readable and understandable by humans.

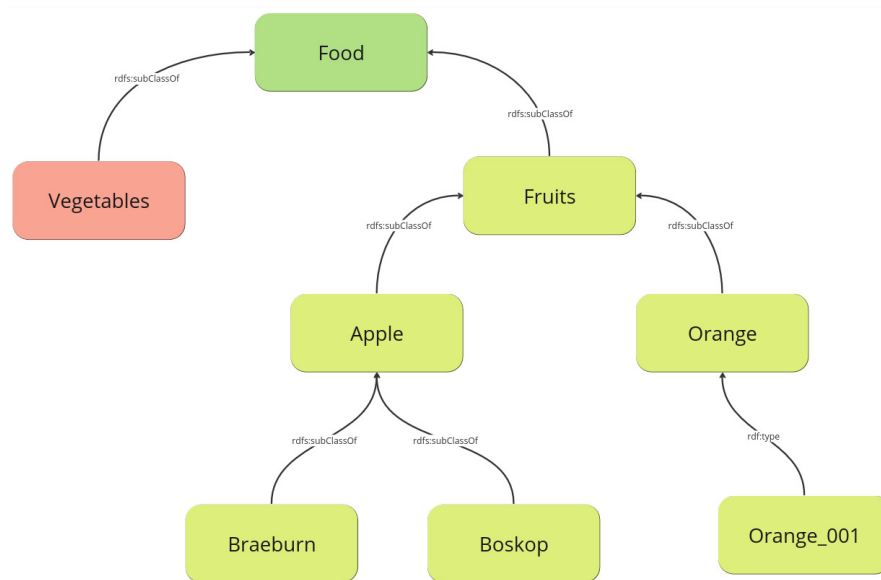


Figure 3.4: Ontology example

Key components of ontologies include:

- **Concepts/Classes:** These represent abstract or concrete entities within a domain. For example, in a food product ontology, *Apple* and *Orange* might be classes.
- **Properties/Roles:** These define the relationships between concepts. For instance, in our example ontology, *subClassOf* could be a property connecting individuals like *Fruits* and *Apple*.
- **Instances/Individuals:** These are specific members or examples of a class, *Or-*

ange_001 can be an instance of the class *Orange*.

- **Axioms:** These are statements that describe the properties and relationships of the entities within the ontology. Axioms help define the logic and rules governing the domain.
- **Hierarchy:** Ontologies often organize concepts into a hierarchy, with more general concepts at the top and more specific ones below. This hierarchical structure aids in categorization and understanding.
- **Inference Rules:** These rules define how new information can be derived from existing information in the ontology. Inferences help systems reason and make deductions based on the knowledge encoded in the ontology.

We utilize ontologies in knowledge representation and reasoning systems to empower the robot with the ability to comprehend and handle information in a structured fashion for our specific objectives.

3.2.2 SWRL

SWRL [18], which stands for Semantic Web Rule Language, is a rule language that allows users to define rules about the relationships between classes and individuals in ontologies represented in the Web Ontology Language (*OWL*). *SWRL* [18] is designed to be used in conjunction with *OWL* to express complex relationships and infer new information based on existing knowledge.

SWRL [18] Rules have a specific syntax and consist of two main components:

- **Antecedent (Body):** This part of the rule specifies the conditions or constraints that must be satisfied for the rule to be applicable. It describes the current state of the ontology that triggers the rule.
- **Consequent (Head):** This part defines the actions or inferences that should be taken if the conditions specified in the antecedent are satisfied. It describes the changes or additional information that should be inferred when the rule is triggered.

SWRL [18] supports various built-in predicates and functions, and users can create their own custom rules to suit their specific ontology. Some common elements in *SWRL* [18] rules include:

- **Individuals:** Refers to specific instances of classes in the ontology.
- **Class and Property Relationships:** Describes relationships between classes and properties in the ontology.

- **Built-in Predicates and Functions:** Includes operations such as arithmetic, string manipulation, and comparison functions that can be used in the rule conditions.

Here's a simple example of a *SWRL* [18] rule:

```
StirringTask(?task) ∧ LiquidIngredient(?ing) ∧ hasIngredient(?task, ?ing)
  ∧ Motion(?motion) ∧ performMotion(?task, ?motion)
  → CircularMotion(?motion)
```

This rule expresses the following statement: Whenever a stirring task is executed with the task having any kinds of liquid ingredients the motion it performs is going to be a circular motion. This rule effectively reclassifies the instance of *Motion* to *CircularMotion*.

SWRL [18] rules are useful for expressing complex relationships and constraints within ontologies, enabling automated reasoning systems to make inferences and derive new knowledge from existing data.

3.3 Software Libraries

3.3.1 RDFLib

RDFLib [19] is a *Python*-package designed to work with *Resource Description Framework* (*RDF*) data. *RDF* is a standard model for data interchange on the web and is used to represent information in the form of subject-predicate-object triples. Key features of the library are:

- **Parsing RDF Data:** *RDFLib* can parse *RDF*-data in various formats such as *RDF/XML*, *N-Triples*, *Turtle*, and more.
- **Creating and Modifying RDF Graphs:** It allows you to create and manipulate *RDF*-graphs, which consist of a collection of *RDF*-triples. You can add, remove, or modify triples in the graph.
- **Querying RDF Data:** *RDFLib* provides a *SPARQL* query engine, allowing you to perform queries on *RDF*-graphs using the *SPARQL* query language.
- **Serializing RDF Data:** You can serialize *RDF*-graphs back into different *RDF*-formats using *RDFLib*, making it easy to store or share *RDF*-data.

- **Working with Ontologies:** It supports working with *RDF*-vocabularies and ontologies, enabling the use of predefined classes and properties from existing ontologies like *RDF*-schema (*RDFS*) or the Web Ontology Language (*OWL*).
- **Integration with Semantic Web Tools:** *RDFLib* can be integrated with other semantic web tools and frameworks, allowing to build applications that leverage semantic web technologies.

3.3.2 Flask

Flask [16] is a lightweight and flexible web framework for *Python*, designed to make it easy to create web applications. It follows the *WSGI* standard and the minimalist approach, offering essential tools and features for building web services and APIs.

3.3.3 Owlready

One important library used for our implementation is *Owlready* [24]. *Owlready* [24] is a *Python*-library designed for ontology-oriented programming. It facilitates the development, manipulation, and querying of ontologies using the Web Ontology Language (*OWL*), a standard for representing knowledge in a machine-readable format. *Owlready* [24] simplifies ontology-related tasks by providing a convenient and object-oriented interface for working with *OWL* ontologies in *Python*.

Key features of the *Owlready* [24] library include:

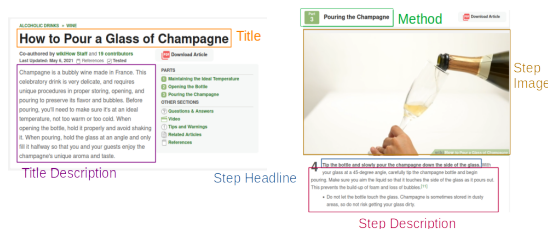
- **Object-Oriented Programming (OOP):** *Owlready* [24] adopts an object-oriented approach, allowing users to interact with ontology entities as *Python*-objects. This makes it more intuitive for developers familiar with *Python*'s principles.
- **Ontology Loading and Parsing:** The library supports the loading and parsing of *OWL* ontologies, making it easy to access and manipulate ontology data within *Python*-scripts or applications.
- **Class and Individual Manipulation:** *Owlready* [24] provides functionality for creating, modifying, and querying classes and individuals within an ontology. This allows for dynamic and programmatic management of ontology content.
- **Reasoning Support:** Depending on the version and features, *Owlready* [24] may offer support for reasoning tasks. Reasoning involves deducing implicit information based on the logical relationships defined in the ontology.
- **Integration with *RDFLib*:** *Owlready* [24] may integrate with *RDFLib*, another

Python-library commonly used for working with Resource Description Framework (*RDF*-data). This integration enhances the capabilities of handling semantic data.

3.3.4 WikiHow-Instruction-Extraction

WikiHow-Instruction-Extraction [4] is an extraction tool, that can collect informations from the WikiHow [3] corpus. The goal of this tool is to analyse a WikiHow corpus using *NLP* techniques to gather information about everyday tasks like *Pouring*, *Cutting* or *Discarding*. These information should support cognitive robots in understanding and parameterizing these tasks to better handle unknown tasks, working in underspecified environments and handling common task-object combinations. Every verb has its own class, in which the verb and the additionally desired hyponyms/synonyms are defined. These verbs serve as keywords for the search in the WikiHow [3] articles. Additionally, various parameters can be set, such as excluding different categories that are not relevant for the search.

WikiHow Article Structure



4 Data Acquisition

In order to create our ontology, we require the knowledge about the mixing behaviour while regarding combinations of ingredients in the context of specific mixing tasks and its variations. In this chapter the required data will be acquired which will lay the foundation of our modeled ontology.

The acquired data needs to be structured in order to model the ontology accordingly. In this chapter we will show the domain knowledge which is derived from the acquired data.

In this chapter we want to introduce the reader to the actual execution on the robot of the motions in a simulation environment. We will illustrate the implemented movements by using the PyCram[2] framework.

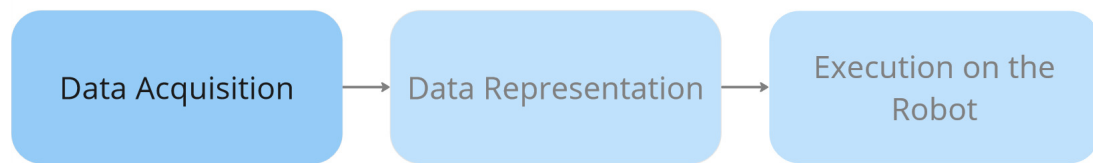


Figure 4.1: Structure Overview: *Data Acquisition*

Data acquisition is the most important step, involving an understanding of what mixing is, how it is performed by people, and how the interactions of different actors in the mixing process can be derived from video analyses. Essentially, it's about discovering the various ways mixing is done, the contexts in which it occurs, and what conclusions we can derive from the data acquisition. This includes identifying different mixing motions under relevant contexts and determining which motions can be regarded or disregarded. First, we have to ask ourselves in the context of this thesis: **What is Mixing?**

What is Mixing? The definition provided by the Oxford Dictionary [14] is as follows:

Definition: To put together or combine (two or more substances or things) so that the constituents or particles of each are interspersed or diffused more or less evenly among those of the rest; to unite (one or more substances or things) in this manner with another or others; to make a mixture of, to mingle, blend.

Ultimately, this definition conveys that mixing requires at least two elements or substances, which are then combined (evenly) with each other, resulting in a (new) substance. This definition is general and can be applied to various contexts. For our work, the aspect of cooking or mixing different (cooking) ingredients is important. Therefore, we consider some hyponyms of the verb *Mixing* irrelevant for our cause and do not take them into account. An adapted definition for our work could be:

Definition: *Mixing* is the combination of various (cooking) ingredients through different motions in a container.

4.1 Task variations

Now that we have defined what *Mixing* means in our context, we aim to identify the various types of *Mixing* and determine which ones are important for our work. We will also specify which types of *Mixing* we will not consider further, providing arguments for our decisions. As our main focus is to represent the knowledge about mixing, first we had to acquire the different types and variations of mixing in order to create a complete knowledge representation. The first step in acquiring the needed data, was to acknowledge which task variations of mixing are actually important. Therefore we had to analyze the word *Mixing* and its hyponyms.

Hyponyms

Hyponyms are subordinated words of a given word, for example one hyponym of *Mixing* could be *Beating*[29]. To conduct this analysis, one can utilize tools from various websites, such as FrameNet[9] and WordNet[35]. These platforms provide users with the ability to search for specific words and obtain various associations for those words, including synonyms, acronyms, or, crucial for our case, hyponyms. A full list of the considered hyponyms can be seen in table 4.1.

WikiHow Extraction

For all those hyponyms, we delegated a *WikiHow* extraction search (see: [WikiHow-Instruction-Extraction](#)) which should show us, how many times one of these words occur, in the context of cooking. To execute the search, we define a new Class *MixingVerb*, which represents the verb *Mixing* and its hyponyms.

For the search, we consider each verb and a triple consisting of the verb forms. We consider three forms of the verbs: the infinitive, the past tense, and the present progressive. An example of such a search would be:

$$\mathbf{MIX} = ("mix", "mixed", "mixing")$$

After defining the set of verbs in *WikiHow-Instruction-Extraction*, we configure this framework to only search for verb occurrences in the category *Food and Entertaining*. This cat-

egory contains articles related to the execution of mixing in the cooking domain, therefore we exclude any other categories from our search.

Hyponyms Occurance

For each defined verb, a search is initiated to determine how often this verb appears in the *WikiHow* [3] articles. Searching for hyponym occurrences helps us, to determine relevant hyponyms, which are relevant for our implementation later on. In the table below the results can be seen.

Hyponym	Occurance
Mix	5300
Amalgamate	0
Beat	956
Blend	1041
Coalesce	1
Combining	3591
Coommingle	0
Compound	0
Conflate	0
Folding	821
Fuse	17
Intermix	0
Join	53
Jumble	0
Lump	7
Merge	6
Pair	352
Stir	6027
Unify	2
Unite	2
Whip	863
Whisk	2267

Table 4.1: *Mix*-verb synoym/s/hyponyms occurance

Further Examination and Conclusion

This search is ultimately intended to provide us with information on which tasks we want to represent in the knowledge base. Each hyponym has to meet the following two criterias: frequency and executability. The first condition is easy to understand; tasks that do not occur or occur very rarely are not considered. The second condition relates to

the executability of the task in the context of robot movements. Additionally, some tasks with relatively high frequency are also examined more closely, as in English, the past tense is used as an adjective under certain circumstances.

Under consideration of the first criteria, the following tasks are not considered: *Amalgamate*, *Coalesce*, *Comingle*, *Compound*, *Conflate*, *Fuse*, *Intermix*, *Join*, *Jumble*, *Lump*, *Merge*, *Unify*, and *Unite*.

The second criteria excludes another task: *Blend*. *Blend* is mostly used in the context of a blending machine, which is not handled by the robot. Without this machine, the blending execution cannot be performed correctly, so this task is excluded for us.

Upon closer examination, we will also not consider the verb *Whip* because it is mostly used as an adjective for ingredients, such as whipped cream. This highlights that only the verb *Whip* has relatively low frequency. The same applies to *Pair*, where the past tense is used to describe a combination of different ingredients, such as wine paired with cheese.

The verb *Combine* is not informative regarding the executed movement of an action and is equated to the verb *Mix*.

In conclusion we consider these tasks relevant for modelling our knowledge base:

- **Mixing Task**
- **Beating Task**
- **Folding Task**
- **Stirring Task**
- **Whisking Task**

4.2 Task Analysis and Definition

All of these tasks need to be analyzed further to understand the context in which they are executed. Our goal is for the system to execute all of our selected tasks in a manner similar to how a human would complete them. We conduct analyses of videos, such as those from *WikiHow* or other sources demonstrating these tasks. In our analysis, we will observe each movement associated with its task. Our results are presented in the table below 4.2. Our analysis includes the executed tasks, used ingredients, and a description of the motion. In the [Appendix](#), we enumerated each analyzed task and provided video references for each task.

Task	Ingredients	Description
Beating	Egg yolk	circular, swirling wildly around the bowl
Stirring	Beaten Egg Yolk , Parmesan and Pepper	Circular, from the inside to the outside.
Whisk	Eggs	Circular but also straight, wildly motion.
Mixing	Eggs, melted butter	Circular, from the inside to the outside, also diving.
Folding	cooked eggs in melted butter	Gently motion from the outisde to the inside straight, then moving about 90 degree before going to the inside again.

Table 4.2: Video analysis

After a thorough analysis of the videos and the information provided in them, we conclude that the movement performed is not only related to the task, but is also influenced by the ingredients used. However, some tasks are deterministic in the sense that the movement is performed independently of the specific ingredients.

In the following, we aim to structure the extracted information from the videos and present our findings.

Video Analysis Conclusion and Results

Based on the extracted information, we conclude that for our goals, the following aspects, in addition to the task, are important and will be further considered:

- **Ingredients:** The ingredients play a crucial role deriving motions associated with the task to be executed. This is going to be explained.
- **Tools:** We don't derive a motion to be executed from tools. We ignore electrical tools, every motion we are going to present can be performend with any tool.
- **Container:** Similarly to tools, the container won't play a crucial part which motion has to be performed, but the range of motion is limited by the container.
- **Motions:** The motions ultimately represent the movement of the robot for our implementation. These movements are extracted from the videos and defined in alignment with robot motions.

Ingredients

Through the video analysis, we conclude that the type of ingredients is very important. Primarily, we divide ingredients into two main categories: *Dry* and *Wet* [26]. This division becomes crucial, especially regarding the measurement of quantities for each type of ingredient.

For our case, a somewhat finer categorization of ingredients is necessary to map the various movements sensibly to the given types. In addition to *Wet* ingredients, we introduce 2 new sub-categories: *Liquid* and *Semi-Liquid* ingredients. This corresponds to ingredients falling under the *Wet* ingredients but with a liquid or semi-liquid state, such as milk, water, eggs, butter and oil. This is significant because some movements differ when the given ingredients are *Semi-Liquid* or *Liquid*.

We will also include 2 sub-categories for the *Dry* category, namely the *Solid* and *Powder* ingredients category. This differs from the each other in that it consists of solid components, while we define the *Powder* category to include powdery substances. The *Solid* category encompasses whole foods like vegetables, fruits, and meats. Through these distinctions, we can create a definition refining the ingredient categories.

Our initial set of ingredients is: *Milk, Oil, Water, Vinegar, Vanilla Extract, Sauces, Egg White, Egg yolk, Butter, Whipped Cream, Flour, Salt, Sugar, Baking Soda, Cocoa Powder, Onions, Pork, Chicken, Minced meat, Bacon.*

These ingredients are commonly used in the analyzed videos and are also very common ingredients for baking and cooking recipes. In the chapter [Data Representation](#) these classes will be categorized.

Containers and Tools

Throughout the video analyses common tools and containers were identified during the execution of various mixing tasks. The most dominant occurring container was a bowl, followed by a pan, cup and then lastly a mug. Those objects can be used to store and serve food for everyday meals. Thus these objects can be summarised into a single concept - *Crockery* [28]

Common occurring tools like a mixer, spatula, spoon and whisk were used. To acquire different kinds of tools and containers, we look into ontologies like *SOMA-Home* [10], where an hierarchy of containers and tools is defined. *SOMA-Home* is a taxonomy modelling concepts for a kitchen environment. *SOMA-Home* uses the class **Crockery**, the class of all kinds of containers. Via subsumption over **Crockery**, we acquire different kinds of containers for mixing.

Identified tools from the video analysis are partially represented by *SOMA-Home*, where mainly cutleries is defined in that ontology. Some tools like mixers and whisks neither belong to the class of all cutleries nor are they represented in *SOMA*. Retrieving a set of tools for these exceptions can't be achieved.

To summarize, the following tools where identified:

- **Cutlery**
- **Mixer**
- **Whisk**

Containers are summarized into the concept of crockery.

Motions

From these videos, we extracted information about the executed motions. In this section, we will describe what the extracted motions are. Additionally, an abstract description is provided of how each motion is implemented in *PyCRAM*, using formulas and plots for visualization. We hold the following assumptions about each motion:

- Each container has a symmetric shape in the x and y directions, considering z as pointing either upwards or downwards.
- The height of the executed motion remains constant.
- There is no collision detection.

These motions were extracted during analysis of the videos:

1. **Circular Motion**
2. **Folding Motion**
3. **Horizontal Elliptical Motion**
4. **Whirlstorm Motion**
5. **Circular Diving To Inner Motion**

Circular Motion The circular motion is generated on a circular plane, which effectively moves the tool in a circle. By having a center coordinate = $(centerX, centerY)$, which is the center coordinate of the container to execute the Circular Motion, we generate a set of coordinates with the following functions:

$$x = centerX + radius * \cos(radian)$$

$$y = centerY + radius * \sin(radian)$$

We retrieve the respective radians using the following function:

$$radian = angle * \frac{\pi}{180}$$

Generating a sequence of points lying inside a circle is accomplished by generating a sequence of angles starting from 0 to 360 degrees, converting them into radians, and using formulas to compute the circular coordinates.

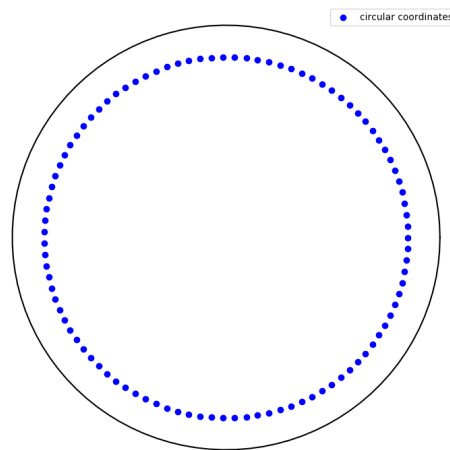


Figure 4.2: Top Down View - Circular Motion

Folding Motion The folding motion differs from circular motion; instead, it follows a linear trajectory. This movement consists of lifting part of the ingredients and gently folding them over the rest. A thorough mixing of the ingredients is possible because of this motion. Repetition of this method ensures even distribution throughout the mixture.

In its current implementation, the motion follows a singular line. One endpoint rests on the edge of the container, while the other is positioned at the center. Although the motion itself is a straight line traversing the container, it is executed by moving from one end to the center and then repeating the motion in reverse—from the center to one of

the ends, creating a folding effect. While this line covers only part of the container, its direction needs realignment to mix different sections of the container thoroughly

This can be accomplished by a 2D transformation, constructing a 2D rotation matrix for some angle θ , where θ determines the rotation of the line.

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

and applying the transformation to the line using this function:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = R(\theta) * \begin{bmatrix} x \\ y \end{bmatrix}$$

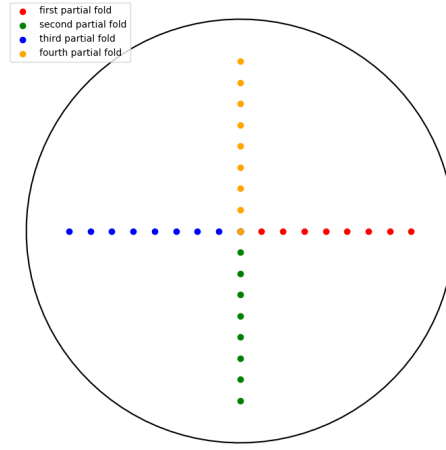


Figure 4.3: Top Down View - Folding 4 areas of a container

In this example, the folding motion is applied to four different sections of the circular container. The number of areas covered or the angle at which the folding line is adjusted is not critical. The fundamental movement remains consistent: the motion proceeds along a line and then reverses.

After executing the folding motion to cover various parts of the container, we apply a transformation with a different angle ϕ .

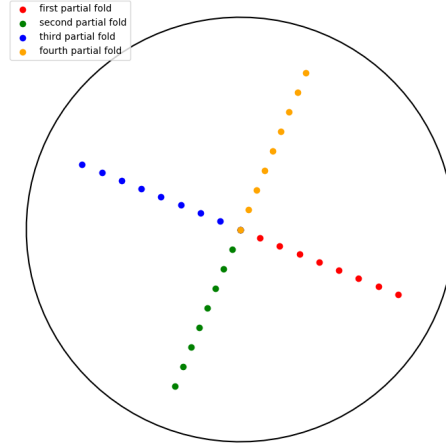


Figure 4.4: Cover different areas of the container with $\phi = 22.5$

Horizontal Elliptic Motion The horizontal elliptic motion is executed on a horizontal plane and involves multiple ellipses to cover the entire container. This motion is commonly used in beating tasks, where ingredients are mixed with rapid, fast motions that resemble quick stirring or beating.

Using the function to compute x,y coordinates on a circle, we use two different radii. One radius is a small value that is not zero, to make each ellipse narrow. The smaller that radius, the narrower the ellipse becomes. The other radius is sampled from an interval. The interval is defined as

$$radii = [radius, \frac{radius}{2}]$$

meaning the sampled radius ranges from the full radius of the container to half of that radius. Defining this interval helps us fit ellipses at the edges of the container, making them smaller.

A sampled radius from this interval is taken, if the following condition is true for all resulting circle coordinates:

$$\sqrt{(xCord - xCenter)^2 + (yCord - yCenter)^2} < radius$$

The end result will be ellipses that are narrow on one axis and conform to the shape of the container on the other axis. A generated ellipse considering the above conditions can look like this:

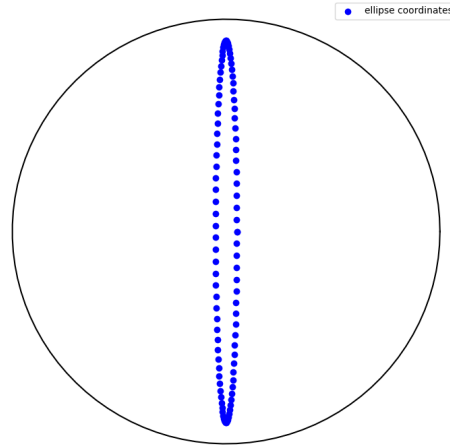


Figure 4.5: Top Down View - Single Ellipse

To cover most areas of any container and assuming that generating y-coordinates used a constant radius, all y-coordinates are shifted after generating the ellipses. To check if the y-coordinates are within the container's bounds, the following condition is checked for every y-coordinate:

$$\sqrt{(yCord)^2 + (yCenter)^2} < radius$$

If this condition is met, the direction of the shift is reversed, switching from incrementing to decrementing and vice versa.

One horizontal elliptic motion consisting of multiple ellipses could look like this:

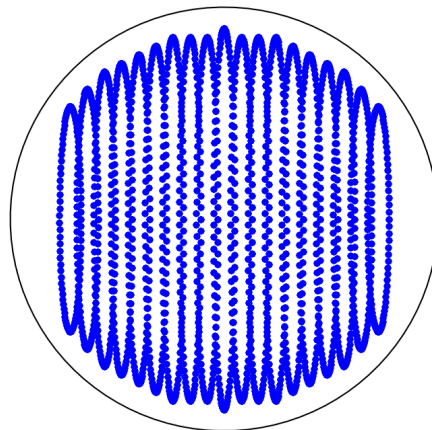


Figure 4.6: Multiple ellipses covering the containers area

Whirlstorm Motion The whirlstorm motion uses the function to generate circle coordinates with differing radii sampled from an interval:

$$[upperBoundRadius, lowerBoundRadius]$$

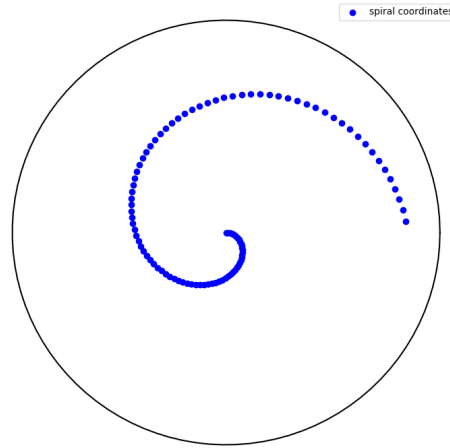


Figure 4.7: Multiple ellipses covering the containers area

The whirlstorm motion is a dynamic and complex swirling pattern that simulates a series of interconnected spiral motions. This motion starts outwardly gradually moving to the inner part of a container, creating a visual effect reminiscent of a whirlpool or tornado. The movement alternates in direction and shape periodically, ensuring a diverse and comprehensive coverage within the containers bounds. This results in a fluid, spiraling trajectory, effectively mimicking natural swirling motions.

In total four whirlstorm patterns are generated, where the first two pattern look like this:

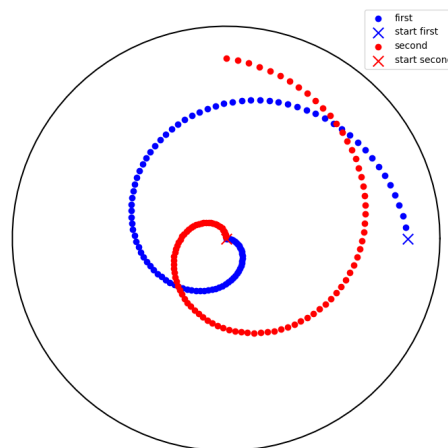


Figure 4.8: Multiple ellipses covering the containers area

The third and fourth look like this:

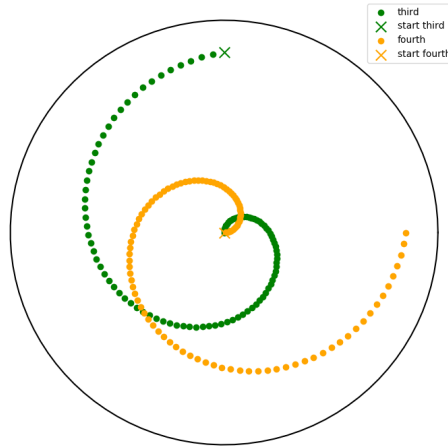


Figure 4.9: Multiple ellipses covering the containers area

Circular Diving To Inner This motion is a circular motion, where the tool is dipped into the container, causing the ingredients to be stirred up and down. This creates a consistent mixture. However, since dipping is involved, which directly contradicts our assumption that height remains constant over the course of the motion, and eventually leaves the boundaries of the used container, we don't provide an abstract implementation of this motion. This motion will have a representation in our *mixing* ontology but won't be implemented in *PyCRAM*.

4.3 Data Acquisition Conclusion

The data acquisition lays the foundation to create a model, mainly an *OWL* ontology, to define relationships between the identified actors for various mixing tasks. By identifying several tasks which were executed in the *WikiHow* videos [3], we were able to define specific motions from different kinds of mixing tasks.

The motions that need to be executed are heavily dependent on the task and the set of ingredients used for mixing. Tools and containers are more or less negligible for the execution of the motions. However, they are still considered, since each container can store different kinds of food, and tools are limited to the container being used.

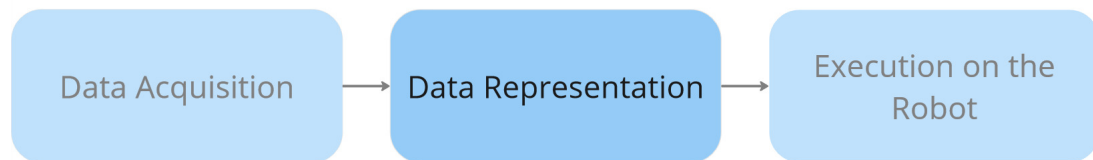
In total five different motions were identified, which will be executed onto four subcategories of ingredients. In the following chapter we will present a taxonomy of mixing with all relevant actors, which enables the robot to execute various mixing tasks in simulation.

5 Data Representation

In order to create our ontology, we require the knowledge about the mixing behaviour while regarding combinations of ingredients in the context of specific mixing tasks and its variations, In this chapter the required data will be acquired which will lay the foundation of our modeled ontology.

The acquired data needs to be structured in order to model the ontology accordingly, In this chapter we will show the domain knowledge which is derived from the acquired data.

In this chapter we want to introduce the reader to the actual execution on the robot of the motions in a simulation environment. We will illustrate the implemented movements by using the PyCram[2] framework.



In this chapter we will present you the mixing knowledge base using *OWL* as basis to formally define knowledge. This knowledge base defines domain knowledge about mixing, which encompasses several aspects derived from the data acquisition. Using assertional knowledge such as the task to execute, the set of ingredients as input for a mixing task, and additional knowledge like which container to use, the knowledge base can infer motions from *SWRL* rules using a reasoner capable with reasoning over these rules. Our goal is not to teach a robot how to execute a specific mixing motion, but rather infer a motion and relevant parameters for this motion.

5.1 The Knowledge Base

Our knowledge base consists of the superclasses *Ingredient*, *DesignedTool*, *Task* and *Motion*. These are the concepts which we derived from the [Data Acquisition](#). The following image illustrates the *Mixing* ontology hierarchy.

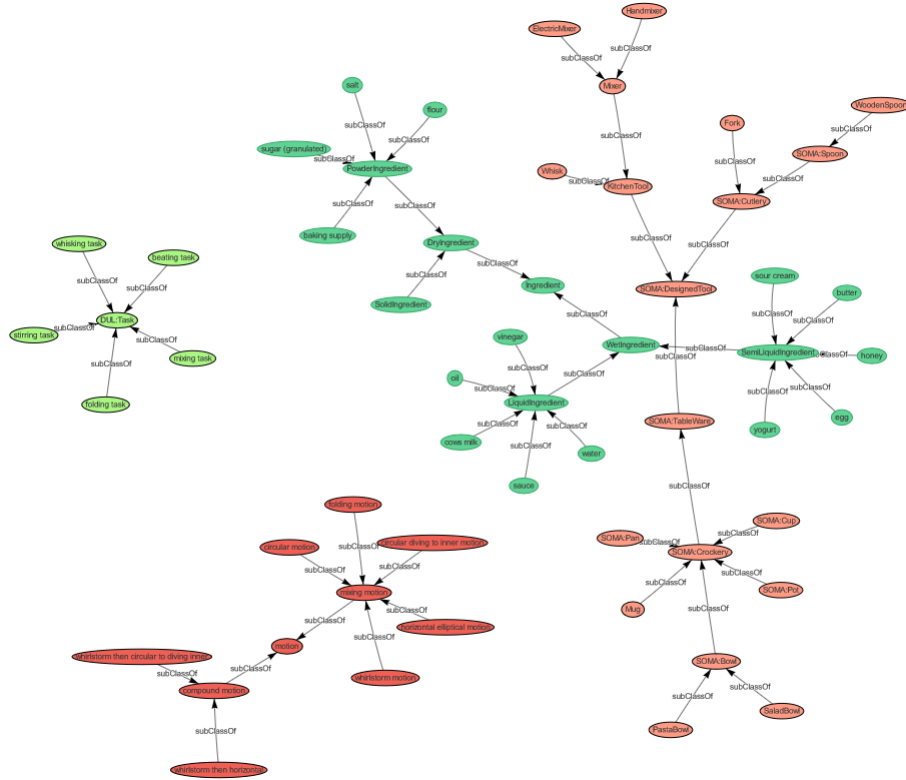


Figure 5.1: The *mixing* Ontology Taxonomy

5.1.1 Ingredients

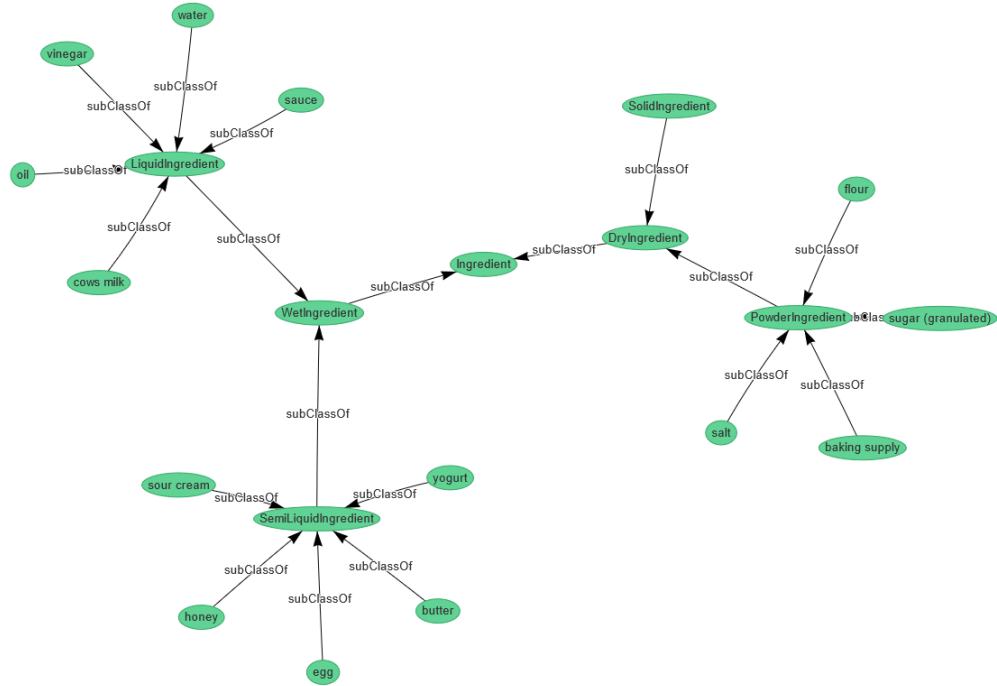


Figure 5.2: Ingredients Hierarchy in the *mixing* Ontology

Ingredients are primarily divided into *Dry* and *Wet*, with a need for finer differentiation. Wet ingredients can be further differentiated into *Liquid* and *Semi-Liquid*, where liquids include examples like water and semi-liquids include examples like egg whites or yolks. Dry ingredients, on the other hand, are divided into the subcategories *Solid* and *Powder*. Solid ingredients include whole vegetables and meat, while powder ingredients are not solid but rather granulated. This hierarchy of ingredients is modeled with a view towards the cooking domain and enables us to formulate rules to infer motions to perform. There does not exist a single hierarchy of ingredients, but rather many hierarchies modeling different aspects of ingredients.

- **Liquid:** *Milk, Oil, Water, Vinegar, Vanilla Extract, Sauces.*
- **Semi-Liquid:** *Egg White, Egg yolk, Butter, Whipped Cream.*
- **Powder:** *Flour, Salt, Sugar, Baking Soda, Cocoa Powder.*
- **Solid:** *Onions, Pork, Chicken, Minced meat, Bacon.*

For each specific ingredient, a respective *FoodOn*[15] class is used. By using *FoodOn*, we

directly include all available subclasses of the *FoodOn* class once the ontology is imported into the *mixing* ontology. Different variations of the same ingredient are then covered in case a finer differentiation is needed. Extending the hierarchy of ingredients can be done by for example integrating *FoodOn* classes.

5.1.2 Tools and containers

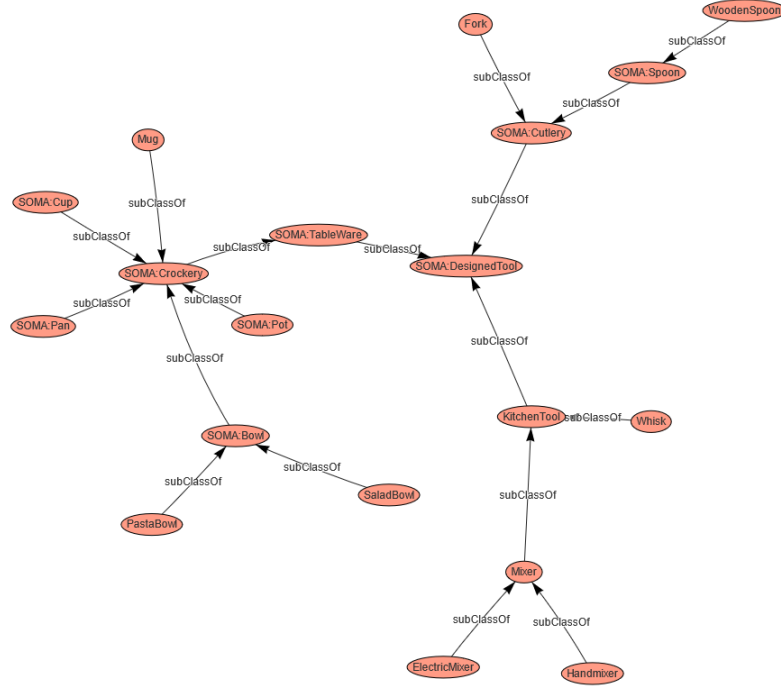


Figure 5.3: Tools and Container Hierarchy in the *mixing* Ontology

Using the identified tools and containers from the data acquisition, we partially use *SOMA* [10] to define parts of the containers and tools hierarchy. We use the class *DesignedTool* from *SOMA-Home*, which consists of all kinds of container and tool classes. *Crockery*, which is subsumed by *DesignedTool*, is included with each container we identified. To include all kinds of cutlery, we use the class *Cutlery*, which subsumes the classes *Fork* and *Spoon*. Mixers and whisks are not represented in *SOMA-Home*, so we introduce a new class, *KitchenTool*, which includes both of these tools.

Using *SOMA-Home* has a few advantages:

1. Using an existing hierarchy can be reused in other taxonomies.
2. If *SOMA-Home* is extended with new containers and tools, these concepts can be easily included.

3. The *mixing* ontology can be used by *Soma-Home*, because both ontologies share classes.

5.1.3 Tasks

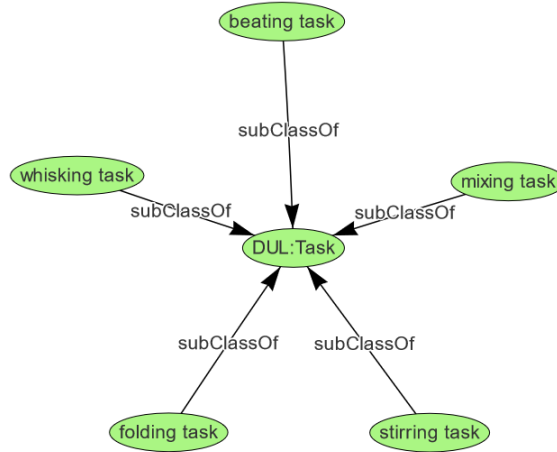


Figure 5.4: Task Hierarchy in the *mixing* Ontology

Mixing tasks are subclasses of the *Task* superclass, where *Task* is a *DUL* concept. All variations of mixing tasks, which were acquired during our data acquisition, are included in the superclass *Task*. These tasks include *Stirring*, which mostly involves a circular motion, *Beating*, which is mainly used in the context of eggs and other wet ingredients, *Folding*, which represents a gentle type of mixing, and *Whisking*, which is similar to beating. While some of these tasks have only one motion associated with them, like the folding task, we discovered that all other tasks can have multiple motions associated with them. This decision is influenced by the set of ingredients on which the motion will be performed.

5.1.4 Motions

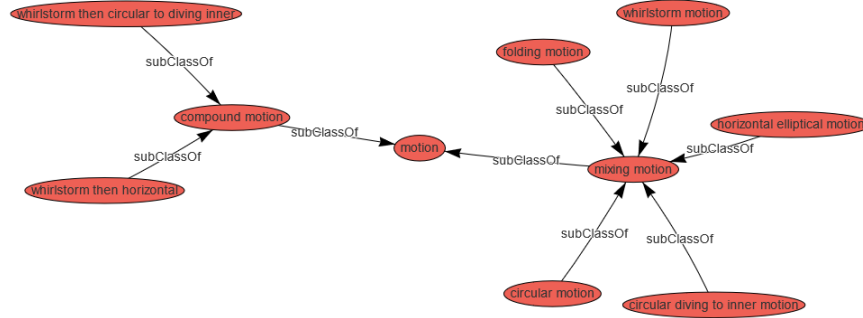


Figure 5.5: Motion Hierarchy in the *mixing* Ontology

The top level class *Motion* is the class of all possible motions which can be performed. *MixingMotion* a subclass of *motion*, are all mixing motions which were found during data acquisition. To make a distinction between one singular mixing motion and motions consisting of at least two motions we created the class *CompoundMotions*, which defines a sequence of motions to be performed. *MixingMotion* is comprised of the following motions:

- **Circular Motion:** Simple Circular Motion
- **Circular Diving To Inner:** Circular Motions including diving moving to the center of a container
- **Folding Motion:** Motion executed as line with back and forth on multiple areas inside the container.
- **Horizontal Elliptical Motion:** Multiple ellipse motions around the container, which resembles mixing ‘wildly’
- **Whirlstorm Motion:** Whirlstorm or spiral like motion

Each motion has class restrictions with different kinds of motion parameters.

The following motion parameters are modelled in the *mixing* ontology:

- **Radius lower Bound Relative:** The smallest possible radius relative to an object.
- **Radius Upper Bound Relative:** The maximum possible radius relative to an object.

- **Folding Rotation Shift:** An angle value of how much the folding line has to be rotated around .
- **Repetitive Folding Rotation Shift:** After the folding motion has been executed this parameter rotates all folding lines. to cover similar areas of a container.
- **Ellipse Shift:** How much the ellipse has to be moved in meters. For example, if the ellipse shift is 0.04, then the ellipse is shifted 4 cm in the positive or negative direction along the x or y axes.

CompoundMotion is comprised of the following classes:

- **WhirlstormThenCircularDivingToInner:** Execute whirlstorm motion followed by circular diving to inner motion.
- **WhirlstormThenHorizontal:** Execute whirlstorm motion followed by horizontal elliptical motion.

Each of these compound motions have an OWL expression stating which motion is executed and which motions if following:

```

Motion
and (hasMotion some
(WhirlstormMotion
and (followedBy some HorizontalEllipticalMotion)
and (followedBy exactly 1 Motion)))

```

This compound motion is equivalent to a *Motion*, which has a motion *WhirlstormMotion* followed by *HorizontalEllipticalMotion*, where *WhirlstormMotion* is exactly followed by one motion.

How each of these motions is related to a task and the set of ingredients will be explained in the next section.

5.2 Rules

In order to infer the right motion based on the ingredients and task input, rules have to be defined. These rules are [SWRL](#)-rules. In this section we want to illustrate the inference on a high level. The rules can be thought of as conditions, which will then result in a motion, for example, if we regard the combination of the task *Mixing* and the ingredient type *Liquid*, we infer the motion *Whirlstorm Motion*. This can be done for every task and

ingredient combination. The inference can be illustrated with decision trees which will be shown in the following section for every task and ingredient type combination available.

5.2.1 Mixing

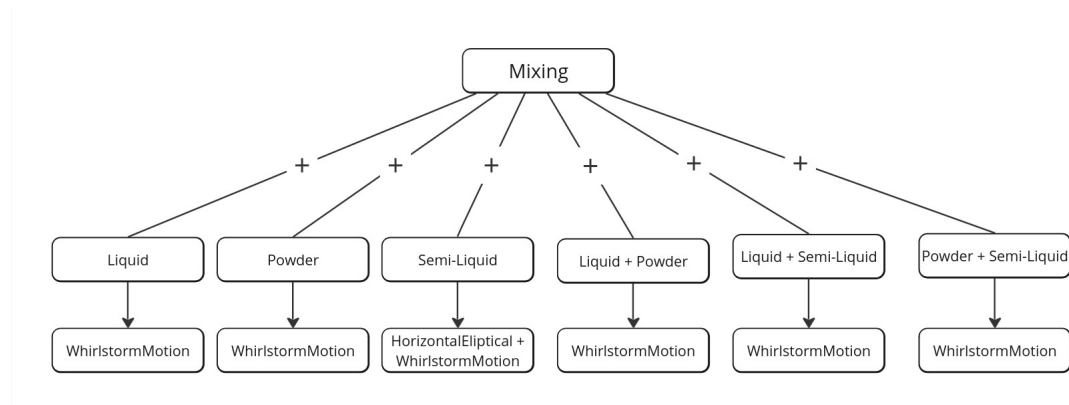
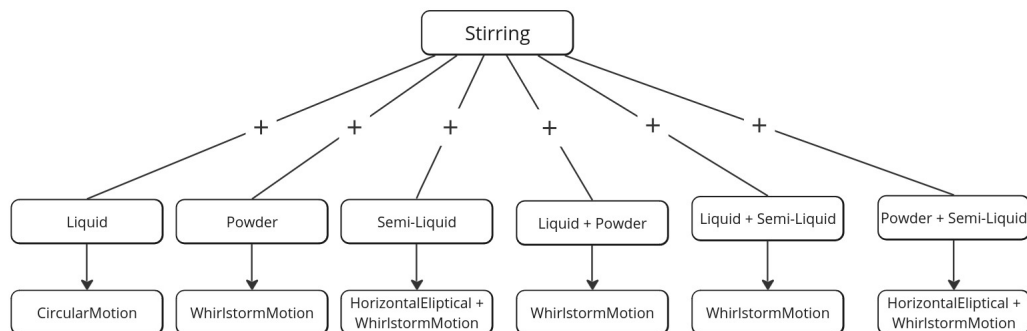


Figure 5.6: Mixing decision tree

Definition: In the context of baking or cooking, a mixing task refers to the process of combining multiple ingredients thoroughly to create a homogeneous mixture. The goal is to distribute the ingredients evenly, ensuring that each component contributes to the overall texture, flavor, and consistency of the final dish or baked good. Mixing is a fundamental step in many recipes and is essential for achieving a balanced and cohesive result.

As can be seen from the illustration, the mixing task can primarily be mapped to the Whirlstorm motion. This is not surprising when considering the definition, as this motion results in the uniform distribution of all ingredients in the container.

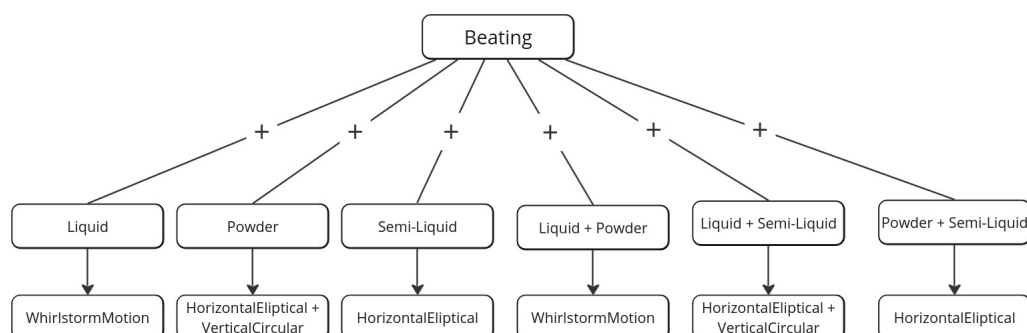
5.2.2 Stirring



Definition: In the context of baking or cooking, a stirring task involves using a utensil, such as a spoon, spatula, or whisk, to agitate and circulate the ingredients within a mixture. The purpose of stirring is to achieve a uniform distribution of ingredients

In comparison to *Mixing*, the *Stirring* task, depending on the ingredients, maps to a broader range of motions. In addition to the *Whirlstorm Motion*, the *Circular Motion* is used here for the first time. This is particularly important when considering the *Stirring* task with the ingredient type *Liquid*, as one does not want the ingredients to be whirled, as would be the case with the *Whirlstorm Motion*, but rather just stirred.

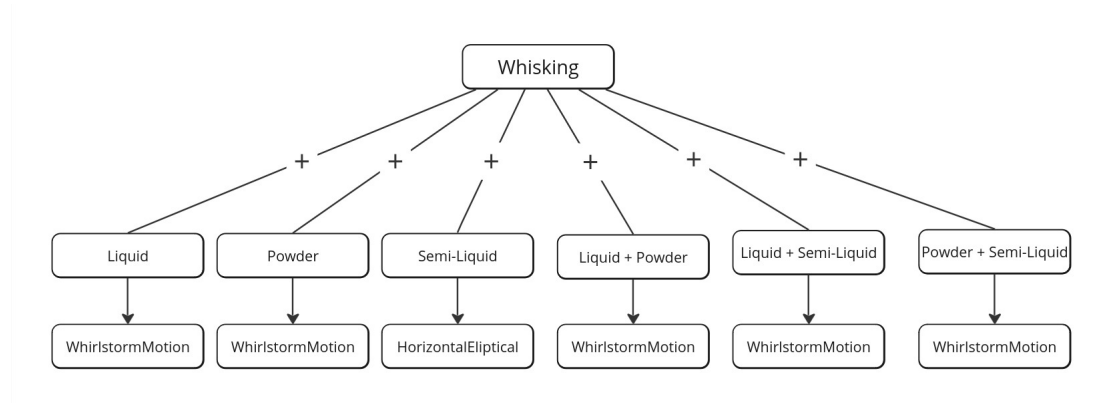
5.2.3 Beating



Definition: In the context of baking and cooking, a *Beating* task refers to the process of vigorously stirring or mixing ingredients to achieve a specific texture or consistency. *Beating* is often done to incorporate air into the mixture, create smooth and uniform blends, or alter the physical properties of certain ingredients.

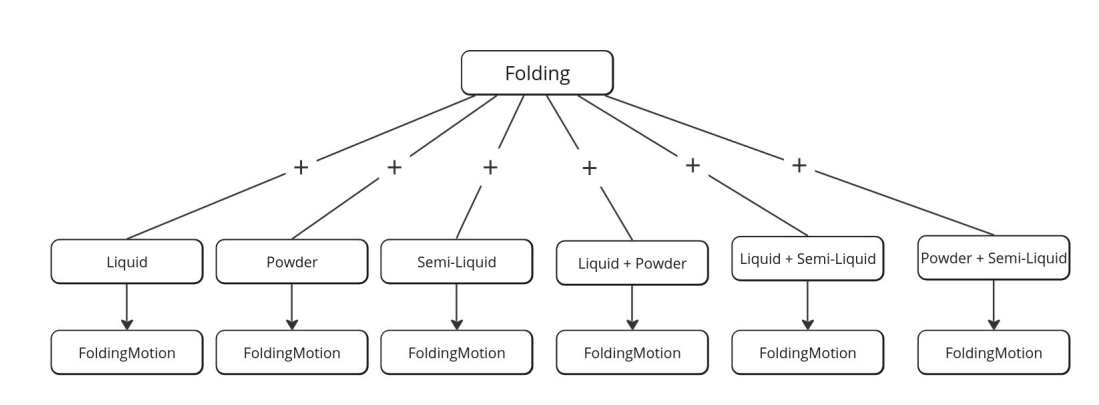
In addition to the *Whirlstorm Motion*, the *Horizontal Elliptic Motion* also predominates here. This can be derived from the definition as well, as this motion requires a wild mixing style, to which the *Horizontal Elliptic Motion* aligns.

5.2.4 Whisking



Definition: In the context of baking and cooking, a *Whisking* task involves using a kitchen utensil called a whisk to mix, blend, or beat ingredients. A whisk typically consists of wire loops or a coil attached to a handle, and it is designed to incorporate air into mixtures, break up clumps, and create a smooth and uniform texture.

5.2.5 Folding



Definition: In the context of baking and cooking, a *Folding* task refers to a gentle mixing technique used to incorporate ingredients without deflating or destroying the air bubbles that have been created. *Folding* is often employed when combining a lighter mixture (such

as whipped cream or beaten egg whites) with a denser one (such as a batter or a heavier mixture). The goal is to maintain the desired texture, lightness, or fluffiness in the final dish.

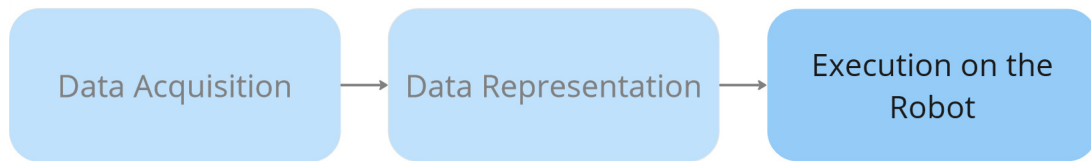
Theoretically, we wouldn't need to graphically represent the *Folding* Task since each task and ingredient combination maps to a single motion, the *Folding* Motion. The graphic was nevertheless included for completeness. The *Folding* Task requires a specific movement that does not remove the air in the mixture, and any other movement would fail.

6 Execution on the Robot

In order to create our ontology, we require the knowledge about the mixing behaviour while regarding combinations of ingredients in the context of specific mixing tasks and its variations. In this chapter the required data will be acquired which will lay the foundation of our modeled ontology.

The acquired data needs to be structured in order to model the ontology accordingly. In this chapter we will show the domain knowledge which is derived from the acquired data.

In this chapter we want to introduce the reader to the actual execution on the robot of the motions in a simulation environment. We will illustrate the implemented movements by using the PyCram[2] framework.



In our work, we have created a knowledge base that can infer parameters to enable an agent to perform various actions. Since testing these actions on a real robot proves to be challenging, we do so in a simulation. This chapter aims to provide an overview of the [Simulation Environment](#) in which we test our work and present it as a proof of concept. Initially, we will explain the environment, including objects, followed by an overview of the motion primitives. Finally, we will showcase some of our motions (in [Simulated Motions](#)) and discuss the [Simulation to Real-World Gap](#) before evaluating every motion as a proof of concept and drawing a conclusion where we discuss existing challenges.

6.1 Simulation Environment

We utilize the simulation environment *BulletWorld* (3.1.4), which is already integrated into the framework [PyCRAM](#) [2]. In this environment, physics is simulated, which is advantageous for executing the motions we have defined or customized. Additionally, leveraging the *PyCRAM* [2] framework allows us to utilize various existing control primitives, such as *Pick and Place*. Communication between the simulation and framework is facilitated by [ROS](#) [32], which communicates via various nodes, each representing a joint, for instance. Through these nodes, one can obtain information about the joints or transmit commands to modify the parameters of the joints accordingly.

The agent that will execute the various motions in the simulation is the [PR2](#) [1]. This robot model has 2 movable arms, as one hand executes the motion. In addition to the

fact that the *PR2* [1] model is already available in the *PyCram* [2] framework, the *PR2* [1] model also has enough joints and degrees of freedom to execute the motions we have implemented.

The agent operates in a kitchen-like environment, the model of which is already included in the *PyCRAM*-core [2] and therefore particularly suitable for our use case. The kitchen also features elements such as drawers that can be opened, which are part of real world interactions. Additionally, the kitchen model includes a table on which the agent's actions are ultimately executed. Other furniture is also present in this environment, but is not relevant to our use case.

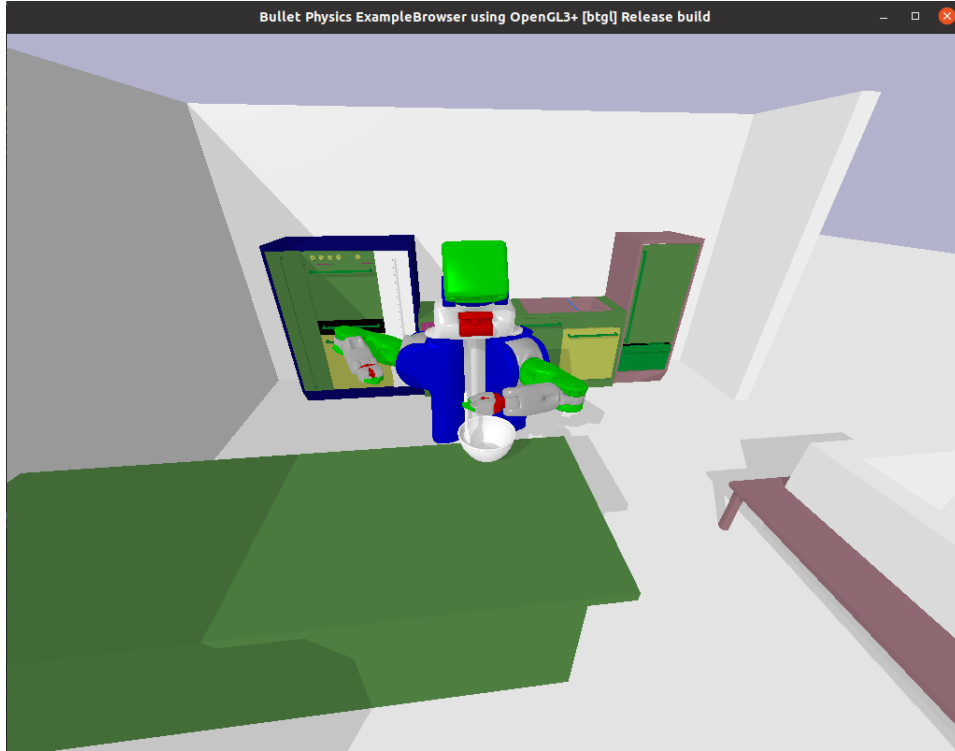


Figure 6.1: Simulation Environment containing the *PR2* and *Kitchen* models.

For the simulation, we use the objects that have also been defined in the ontology (see [Containers and Tools Acquisition](#)). The following objects are available:

- **Container:**

- *Small and Large Bowls* - Defined in the ontology as *SaladBowl* and *PastaBowl*
- *Pan*
- *Pot*
- *Cup*
- *Mug*

- **Tools:**

- *Whisk*
- *Spoon/Wooden Spoon*
- *Fork*



Figure 6.2: Used models for the simulation: *Fork*, *WoodenSpoon*, *Spoon*, *Whisk*, *Mug*, *Cup*, *Pan*, *Pot*, *Small Bowl*, *Big Bowl*

In the simulation environment there are no models of ingredients included. Executing motions in the simulation is done as if the motion is executed in the vacuum, physical

laws regarding ingredient interaction do not apply. The set of ingredients are nonetheless relevant because those ingredients have to be mixed.

6.2 Control Primitives

The framework [PyCram](#) [2] offers a variety of pre-implemented control primitives. These primitives can be used as building blocks to structure the entire execution of a task. These primitives are also defined as *Action Designators*, and the following ones are relevant for our use case:

```
NavigateAction(target_locations=[pickup_pose.pose]).  
    resolve().perform()  
  
PickUpAction(object_designator_description=tool_object,  
    arms=pickup_pose.reachable_arms,  
    grasps=["top"]).resolve().perform()  
  
NavigateAction(target_locations=[nav_pose]).  
    resolve().perform()  
  
LookAtAction(targets=[container_object.resolve().pose]).  
    resolve().perform()  
  
MixingActionSWRL(  
    object_designator_description=container_object,  
    object_tool_designator_description=tool_object,  
    ingredients=[Set of Ingredients],  
    task="given task",  
    arms=["left"],  
    grasps=["top"]).parameters_from_owl().perform()
```

The last part is a custom resolver (see ??), which returns a *Mixing Action Designator* instance to perform the mixing action. This designator receives as parameters a container object and a tool object (both not significant for inference), as well as a set of ingredients and a task, which are significant for inference.

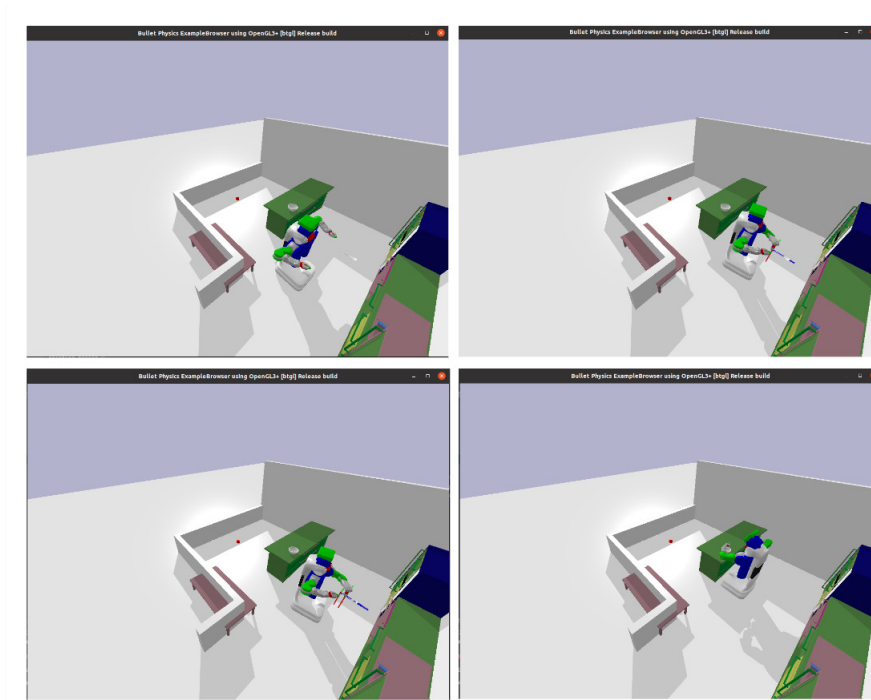


Figure 6.3: Featured *Control Primitives*

6.3 Implementation in PyCram

6.3.1 Robots Plan Execution

The robots plan used for implementing the `MixingActionDesignator` and its evaluation is a simple plan, including the following steps:

- Park Arms
- Elevate the robots torso
- Navigate to the tool used for mixing
- Pick up tool
- Park arms
- Move to kitchen counter
- Look at the container to mix the ingredients

- Mix ingredients

In the following subsections we will explain, how the robot uses its knowledge during task execution, to mix.

6.3.2 Asserted Knowledge

Any robot capable of executing mixing motions need to have knowledge before executing a mixing task. The robot needs to know where the container and tools are located, for picking up and for navigation. The location of mixing is important as well and should be known before executing mixing action. Also, the robot needs to know which ingredients have to be mixed, in simulation only their existence have to be known. It needs to know which task to execute, since we differentiate between different mixing tasks as discussed in this section.

6.3.3 Inferring Knowledge

As of now, the robot doesn't know what kind of mixing motion it has to perform. Its associated parameters are also unknown. Therefore, the robot needs to reason about its assertional knowledge. Once it has access to all relevant information, it can infer the appropriate mixing motion or motions, in case a compound motion has been inferred, with its corresponding parameters through logical reasoning. Logical reasoning is performed inside the custom resolver *MixingActionSWRL*.

6.3.4 MixingActionSWRL

MixingActionSWRL is a custom resolver using our *mixing* ontology to infer motions from the robots assertional knowledge. The resolvers task is to fill the missing descriptions of which motions should be executed with its associated parameters.

Initialization The resolver loads the mixing ontology, creates **NamedIndividuals** of the task to execute and of all ingredients which are being mixed. Since we wanna infer facts from SWRL rules, every actor in the mixing process has to be created as a **NamedIndividual**. All names need to be available for later instantiation. A **NamedIndividual** is created from the high level class *Motion*, representing the individual of all possible motions. The resolver attempts to reclassify this motion into a specific motion that is going to be executed on the robot.

First container, tool and ingredients are instantiated, additionally a task instance is created and its relations with the other individuals are assigned. To find the proper corresponding class, to the names of the container, ingredients, task and tool we perform fuzzy string matching to find a matching class to the name.

Fuzzy String Matching Using the levenshtein distance [37], syntactic similarity of two different strings is computed. A low distance score indicates a higher similarity than a high score. However, this metric does not account for semantic similarities, as the words meanings are not considered in the computation of distance.

Inference Once the relations have been assigned, using *OWLReady* with builtin functions calling reasoners like *HermIt* [30] or *Pellet* [31], reclassifies the motion instance using the defined *SWRL* rules inside the *mixing* ontology. Since the individual instantiated from the class *Motion* has been reclassified the motion parameters can be extracted from the class.

Finally, the resolver completes the sequence of motions to be executed with its associated parameters and forwards it to the *MixingAction*.

6.3.5 MixingActionDesignator

The *MixingActionDesignator* is a newly implemented action designator designed to perform various types of mixing motions or sequences of motions within a specific container using a specific tool. This designator generates poses that follow a trajectory, simulating the process of mixing in the bulletworld environment. To effectively break down the action into executable motions, certain designator descriptions are required.

6.3.6 Implementation

Designator Description

1. Container Object Designator:

- This object designator contains information about its name and its bulletword internal representation. Its bulletworld representations holds information like the current objects pose and its 3D axis aligned bounding box. This bounding box can be used to compute the dimensions of the container along each axis. Knowing the bounds and the pose of the container is crucial, because the motion has to be performed inside the container within its boundaries.

2. Tool Object Designator:

- Analogous to the container this designator contains the same type of information. Knowing the dimensions of the tool is crucial in avoiding collision with the container.

3. Arm:

- Which arm should be used to perform the mixing action

4. Motions:

- A list of motions as strings to be performed.

5. Motion Parameters:

- Each motion has its own key value pairs of parameters. These parameters alongside the motion are resolved by the `MixingActionSWRL`

Resolving motion and parameters The *MixingActionDesignator* calls the *MixingActionSWRL* resolver, to infer the motion with its associated parameters.

Sequential Processing in *MixingActionDesignator* Once all descriptions are available for the *MixingActionDesignator*, it attempts the following steps in sequence:

1. **Retrieve BulletWorld Objects:** The BulletWorld objects for both the container and the tool are retrieved from the object designator. Each BulletWorld object has a 3D axis-aligned bounding box. Using this bounding box, the dimensions can be computed. The implemented function *get_object_dimensions()* returns the three dimensions of each BulletWorld object.
2. **Motions:** Access the motion names and find out which motion has to be executed.
3. **Calculate Radius Bounds:** The upper and lower radius bounds, which are relative values, are accessed to compute the absolute radius for the respective mixing motion, using the dimensions of container and tool, to ensure that any mixing motion is executed inside the container.
4. **Transform Container Pose:** The container's pose is retrieved and transformed into its own coordinate frame from the world coordinate frame to compute mixing poses fitted to the container in its coordinate frame.

5. **Generate Motion Trajectory:** Based on the current motion, 3D coordinates are generated using the functions defined in the section [Motions](#), the values inside the motion parameters list are used for computation.
6. **MoveTCP:** For each generated pose, that pose is elevated by a constant value and the execution of motion is done with MoveTCP, which moves the tool center point of the robots arm.

6.3.7 Simulated Motions

This section will illustrate the results of our implemented motions in the *MixingAction-Designator*. We will contrast each motion with a plot and a picture, which contain the generated poses with their visualized axes.

Whirlstorm Motion

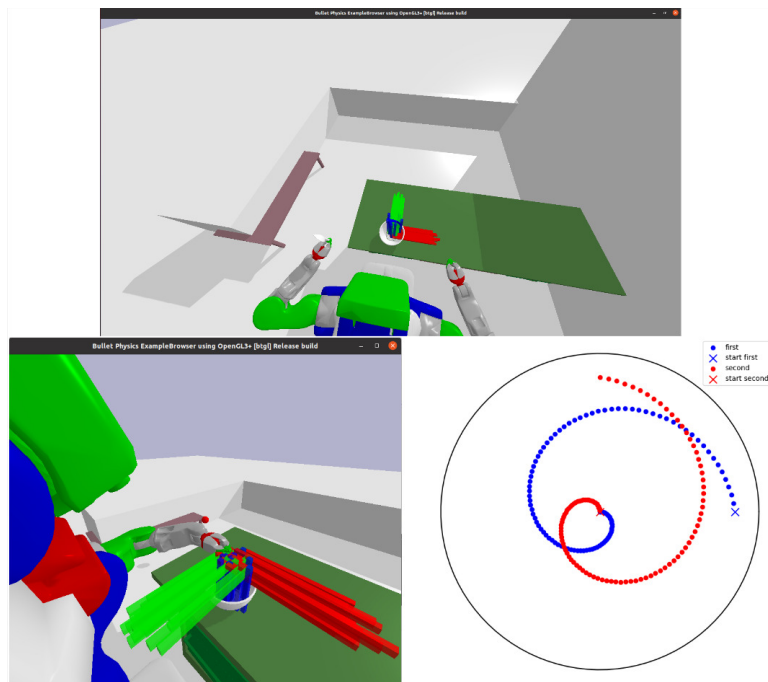


Figure 6.4: Feautured *Whirlstorm Motion*

Circular Motion

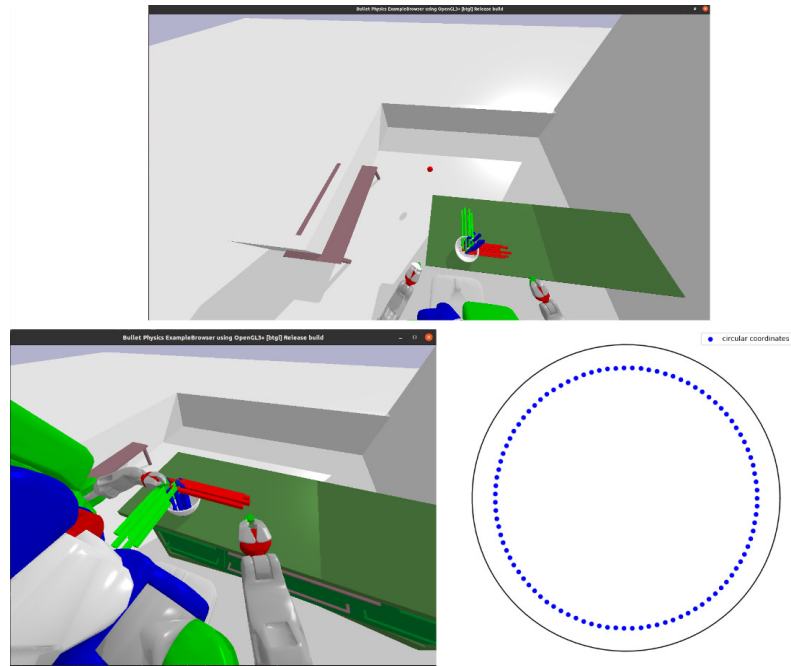


Figure 6.5: Featured *Circular* Motion

Horizontal Elliptical

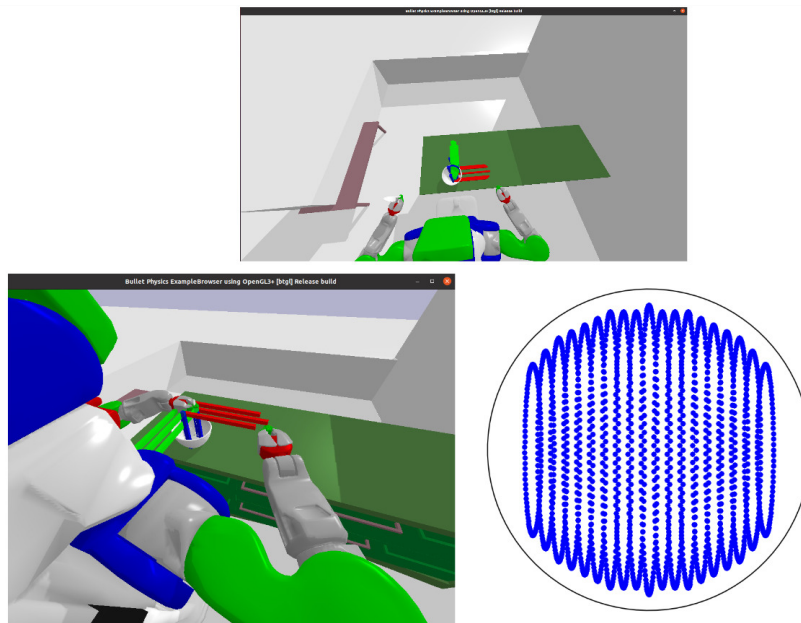


Figure 6.6: Featured *Horizontal Elliptical* Motion

Folding Motion

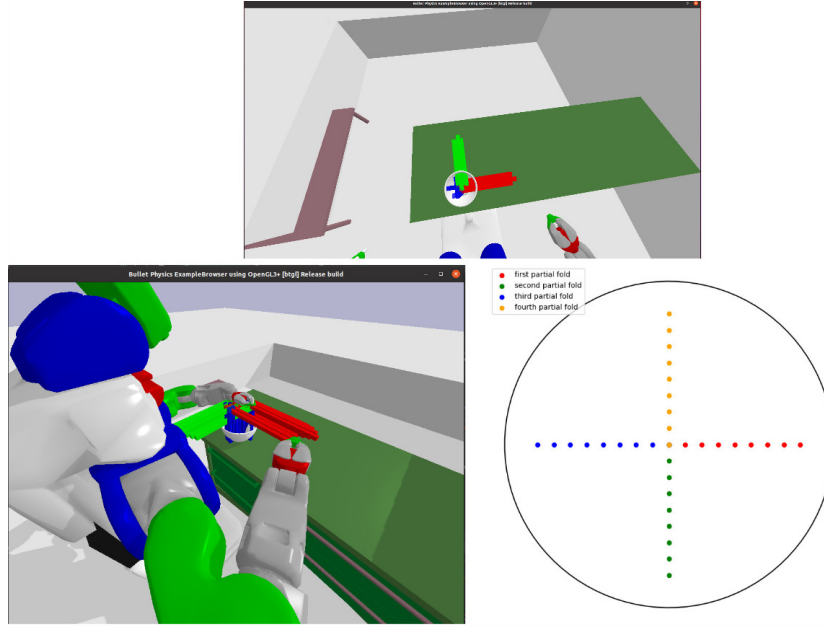


Figure 6.7: Featured *Folding* Motion

6.4 Simulation to Real-World Gap

Robots operating in the kitchen domain face numerous challenges when using the *MixingActionDesignator* in real-world scenarios. Unlike in simulations, where each executed movement of the mixing action follows precise trajectories, the complexity of executing mixing tasks and their associated motions in real-world conditions rises tremendously. The *MixingActionDesignator* lacks any form of failure handling to address potential challenges in real-world scenarios

When mixing different ingredients in the real world, the physical properties of the ingredients must be considered, as the motion can no longer be executed in a vacuum. Properties such as viscosity, density, and texture will influence the ability to mix these ingredients, potentially leading to inefficient mixing and unsatisfactory results for the user. The mixing process is further complicated by the mechanical limitations of the robot, where applied force, speed, and other characteristics will affect the outcome.

A robotic system relying on ground truth loses this certainty when transitioning to the real world, encountering various types of uncertainty instead. The robot loses knowledge about the locations of ingredients, tools, and containers. Consequently, it requires a functioning perception system capable of identifying and locating objects in a scene. Additionally, the computation of object boundaries must be done via approximation, which is a critical

requirement for the *MixingActionDesignator*.

The robot’s ability to perceive all required objects for the mixing task can be limited by visual phenomena such as poor lighting conditions, reflections, occlusions, and other visual challenges. Moreover, the inherent variability in the appearance and shape of ingredients can challenge object recognition algorithms, resulting in misidentification or ambiguity.

In real world scenarios, object poses are no longer known to the robot, making reliable pose estimation essential. A good grasp pose is crucial for all objects to ensure they are picked up properly and transported safely to their destination. For instance, a tool should generally be picked up by the handle rather than the head to ensure proper handling. Without accurate pose estimation, the robot might misalign with objects, leading to failed grasp attempts, dropped items, or even damage to the objects or the robot itself. The variability in object shapes, sizes, and textures requires the robot to continuously adapt its grasping strategy, further emphasizing the need for precise and reliable pose estimation to maintain efficiency and accuracy in the task.

6.5 Challenges and Difficulties, Summary of the Simulation

Overall, we are satisfied with the results of the simulation. In the following section [Evaluation](#), the various motions are tested and evaluated with different combinations of tasks, ingredients, tools, and containers. One point we would like to address is the initially planned motions, which have a height increment, meaning the motions would not only be executed in 2 dimensions but in 3. Implementing this in the simulation proved to be challenging, so we decided against further consideration of these motions due to the effort involved.

At the current state the robot in simulation either successfully executes actions or fails to do so. In case of failures there is no recovery to successfully execute a mixing task. The robot is not able critically reflect upon its made failures.

6.6 Evaluation

In this section, we aim to present our implemented/adapted motions as a proof of concept. We consider all possible combinations of tasks and ingredients applied to every possible combination of tools and containers. Each task will showcase only one example in this chapter, with the rest available and more detailed in the appendix. The respective task trees are located in Chapter [Data Representation](#), which illustrates which motions result from combinations of tasks and ingredients.

Through this evaluation, we aim to demonstrate that the knowledge base we have developed can aid in inferring dynamic motions along with parameters that adapt to both the tool and container. Our parameters are relative to the dimensions of the container, specifically the radius within which the motion is executed, as well as the dimensions of the tools used to execute the motion. It is crucial that the motion in the simulation does not exceed the predetermined dimensions, as this could have potentially serious consequences when applied in the real world.

Mixing Task

Nr.	Ingredients	Tool	Container	Inferred Motion
1	Liquid	WoodenSpoon	Salad Bowl	Whirlstorm Motion
2	Powder	WoodenSpoon	Salad Bowl	Whirlstorm Motion
3	Liquid + Dry	WoodenSpoon	Salad Bowl	Whirlstorm Motion
4	Liquid + SemiLiquid	WoodenSpoon	Salad Bowl	Whirlstorm Motion
5	Powder + Wet	WoodenSpoon	Salad Bowl	Whirlstorm Motion
6	SemiLiquid	WoodenSpoon	Salad Bowl	VerticalCircular + Whirlstorm Motion

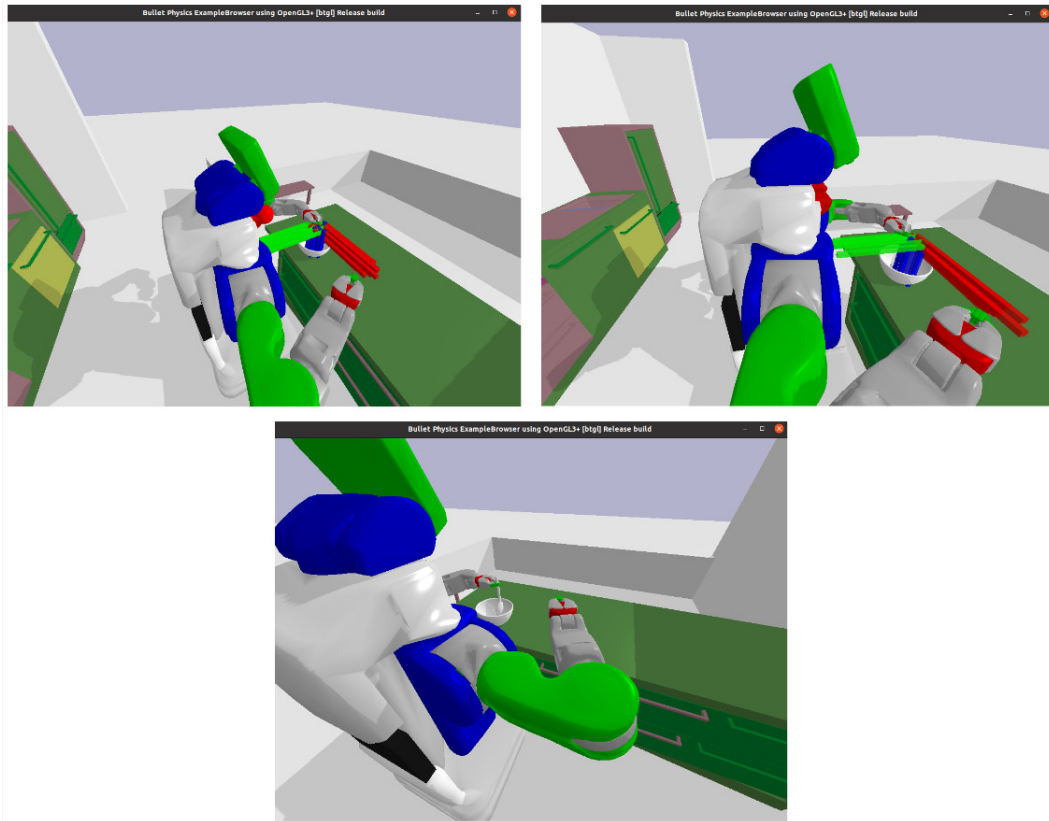
Table 6.1: Mixing Task

- Inferred Parameters for **1-5**:

$\Rightarrow$ radius_lower_bound_relative = 0.0 ,
radius_upper_bound_relative = 0.7

- Inferred Parameters for **6**:

$\Rightarrow$ radius_lower_bound_relative = 0.0 ,
radius_upper_bound_relative = 0.7 ,
ellipse_shift = 0.04

Figure 6.8: Inferred *Mixing* Motions

Beating Task

Nr.	Ingredients	Tool	Container	Inferred Motion
1	Liquid	Fork	Salad Bowl	Whirlstorm Motion
2	Powder	Fork	Salad Bowl	Whirlstorm Motion + Horizontal Eliptical Motion
3	Liquid + Powder	Fork	Salad Bowl	Whirlstorm Motion
4	Liquid + SemiLiquid	Fork	Salad Bowl	Whirlstorm Motion + Horizontal Eliptical Motion
5	Powder + SemiLiquid	Fork	Salad Bowl	Horizontal Eliptical Motion
6	SemiLiquid	Fork	Salad Bowl	Horizontal Eliptical Motion

Table 6.2: Beating Task

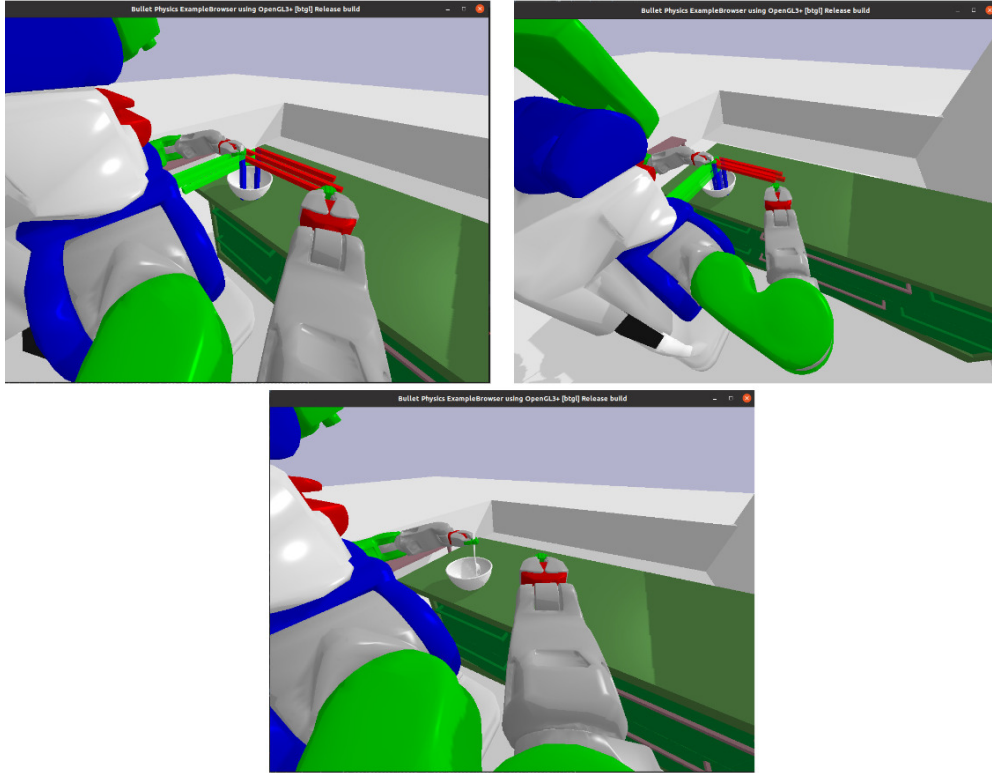


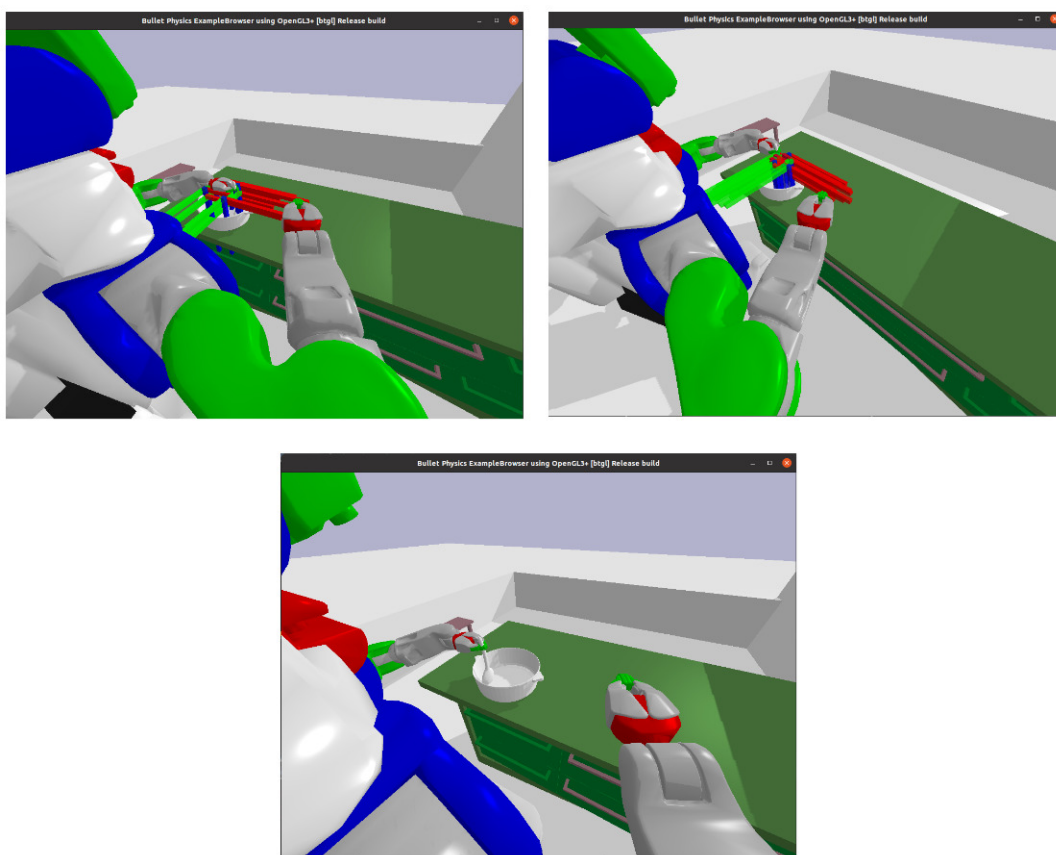
Figure 6.9: Inferred *Beating* Motions

- Inferred Parameters for **1,3**:
 $\Rightarrow$ `radius_lower_bound_relative` = 0.0 ,
`radius_upper_bound_relative` = 0.7
- Inferred Parameters for **2,4,5,6**:
 $\Rightarrow$ `radius_lower_bound_relative` = 0.0 ,
`radius_upper_bound_relative` = 0.7 ,
`ellipse_shift` = 0.04

Stirring Task

Nr.	Ingredients	Tool	Container	Inferred Motion
1	Liquid	Wooden Spoon	Pot	Circular Motion
2	Powder	Wooden Spoon	Pot	Whirlstorm Motion
3	Liquid + Powder	Wooden Spoon	Pot	Whirlstorm Motion
4	Liquid + Semi-Liquid	Wooden Spoon	Pot	Whirlstorm Motion
5	Powder + Semi-Liquid	Wooden Spoon	Pot	Whirlstorm Motion + Horizontal Eliptical Motion
6	Semi-Liquid	Wooden Spoon	Pot	Whirlstorm Motion + Horizontal Eliptical Motion

Table 6.3: Stirring Task

Figure 6.10: Inferred *Stirring* Motions

- Inferred Parameters for 1:

$\Rightarrow$ radius_lower_bound_relative = 0.7,
radius_upper_bound_relative = 0.8

- Inferred Parameters for **2,3,4**:

=> radius_lower_bound_relative = 0.0,
radius_upper_bound_relative = 0.7,
ellipse_shift = 0.04

- Inferred Parameters for **5,6**:

=> radius_lower_bound_relative = 0.0,
radius_upper_bound_relative = 0.7,
ellipse_shift = 0.04

Whisking Task

Nr.	Ingredients	Tool	Container	Inferred Motion
1	Liquid	Whisk	Bowl	Whirlstorm Motion
2	Powder	Whisk	Bowl	Whirlstorm Motion
3	Liquid + Powder	Whisk	Bowl	Whirlstorm Motion
4	Liquid + Semi-Liquid	Whisk	Bowl	Whirlstorm Motion
5	Powder + Semi-Liquid	Whisk	Bowl	Whirlstorm Motion
6	Semi-Liquid	Whisk	Bowl	Horizontal Eliptical Motion

Table 6.4: Whisking Task

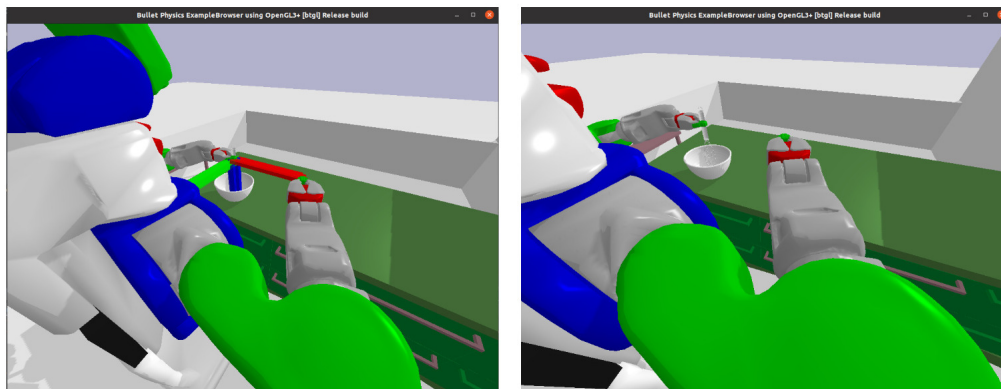


Figure 6.11: Inferred *Whisking* Motions

- Inferred Parameters for **1,2,3,4,5**:

=> radius_lower_bound_relative = 0.0,
radius_upper_bound_relative = 0.7

- Inferred Parameters for **6**:

$\Rightarrow$ radius_lower_bound_relative = 0.0,
 radius_upper_bound_relative = 0.7,
 ellipse_shift = 0.04

Folding Task

Nr.	Ingredients	Tool	Container	Inferred Motion
1	Liquid	Fork	Pot	Folding Motion
2	Powder	Fork	Pot	Folding Motion
3	Liquid + Powder	Fork	Pot	Folding Motion
4	Liquid + Semi-Liquid	Fork	Pot	Folding Motion
5	Powder + Semi-Liquid	Fork	Pot	Folding Motion
6	Semi-Liquid	Fork	Pot	Folding Motion

Table 6.5: Folding Task

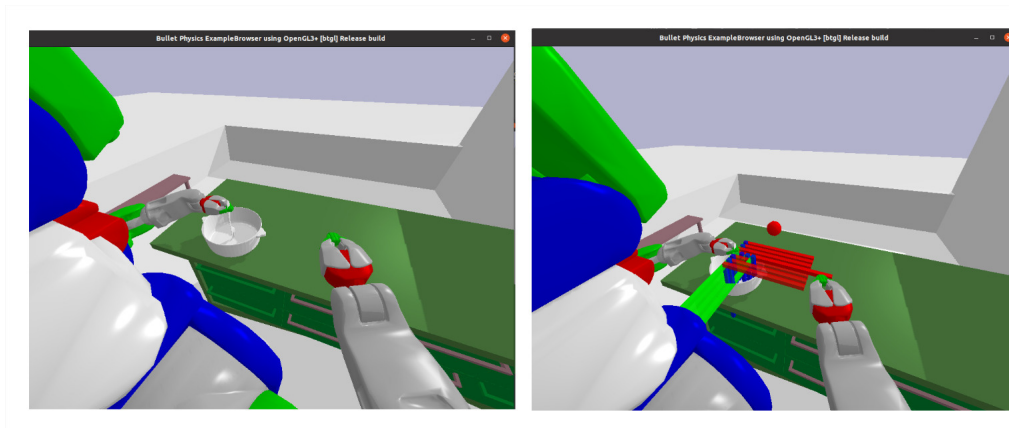


Figure 6.12: Inferred *Folding* Motions

- Inferred Parameters for **1-6**:

$\Rightarrow$ radius_lower_bound_relative = 0.0,
 radius_upper_bound_relative = 0.7,
 folding_rotation_shift = 90,
 repetitive_folding_rotation_shift = 22.5,

7 Knowledge Graph Visualization Tool

Many existing knowledge graph visualizations are restricted to displaying certain types of relationships. Most commonly, a class hierarchy of an ontology is visualized for the user. Other graph visualization tools have built in features capable of displaying relationships between instantiated classes or *NamedIndividuals*. A few of these graph visualizers are mentioned in [Related Work](#).

What we didn't find is a graph visualizer capable of displaying complex class expressions, therefore, we decided to develop our own graph visualizer, which is capable of displaying class hierarchies with complex constructs.

This chapter will present the main features of our *Knowledge Graph Visualization Tool*, where explain explain its capabilities and limitations.

7.1 Main concept

We present you our three main concepts, which are included in the *Knowledge Graph Visualization Tool*:

- **Visualization of Class Hierarchy:** Our primary goal is to visualise a class hierarchy, where class relations are visible to the user and visualise complex class expressions as a part of the corresponding class. This will be done as compact as possible. Furthermore to differentiate between classes and class expressions we aim to make such distinction via different coloring. Additionally, we aim to keep the amount of nodes and edges low, to reduce clutter in the graph, making exploring graph easier and increase performance of visualizing such graphs.
- **Query Builder:** To clarify the relations between the queries, we implement a query builder that, given a class, can point to other classes through the relation, which in turn can point to further classes through another relation, as long as the user desires or there are no further relations. No *SPARQL* knowledge is required query language, making it usable for the user. The *Query Builder* outputs a filtered graph with the selected triples and generates a *SPARQL* query.
- **Rule Based Inference:** Our thesis relies on running inference over *mixing* we implement an inference tool running reasoning over our ontology, to find motions, anal-

ogously to [MixingActionSWRL](#). The result is visualized in a graph and a sequence of actions in natural language are generated, to highlight inferred knowledge.

7.2 Features

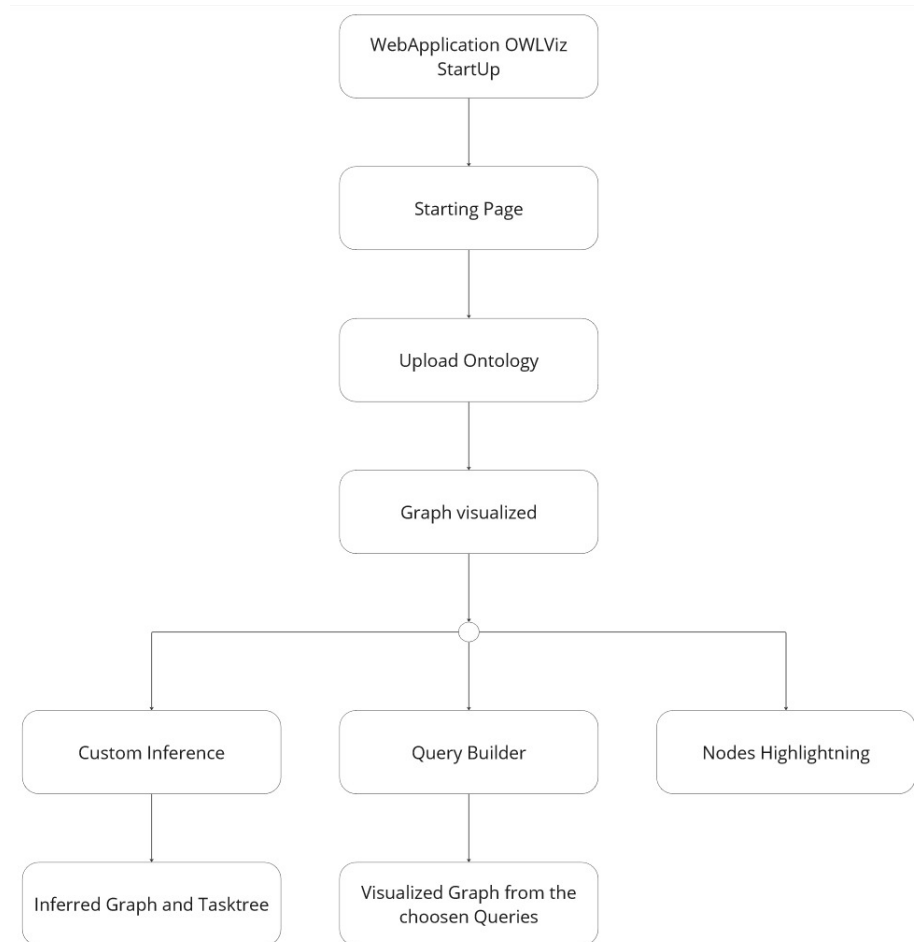


Figure 7.1: Architecture chart for the *Knowledge Graph Visualization Tool* framework

7.2.1 Graph Visualization

In this section, we present a simple example of our framework, from processing the ontology to visualization in the web application. For this purpose, we create a small and simple ontology to explain the basic visualization of an ontolgy. The visualization library *vis.js* [36] has been used to create graph visualization in web-browsers.

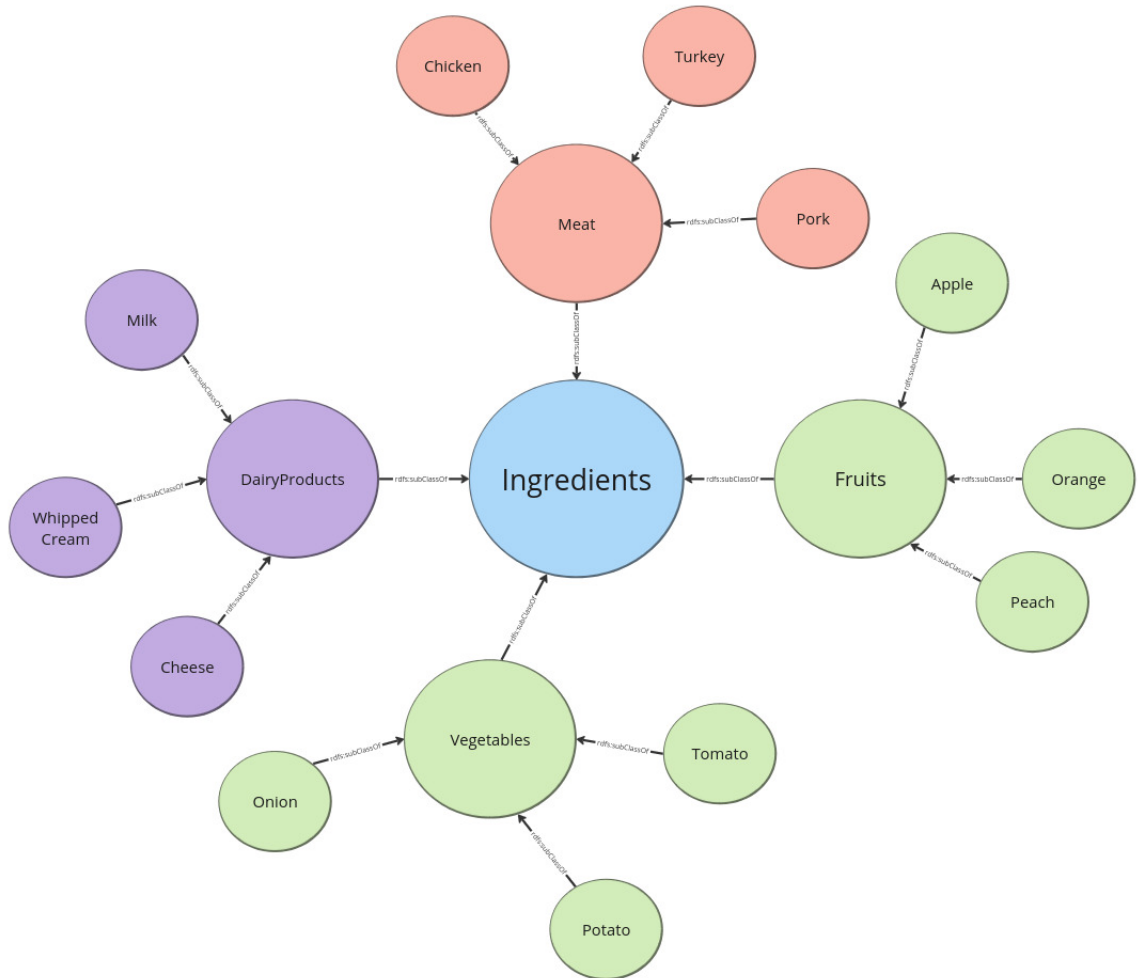


Figure 7.2: Trivial example of an ontology

The ontology created for this purpose aims to provide a simple representation of various food products. It includes superclass categories such as *Fruits*, *Vegetables*, *Meat*, and *Dairy Products*, as well as subclasses like *Apple* and *Orange*, which are subclasses of the *Fruits* superclass. This visualization is a simple visualization of a class hierarchy of food products, where each node is connected via a directed edge labeled with *subClassOf*.

7.2.2 Graph Processing

The core component of the graph visualizer is the visualization of a hierarchy of classes. In this hierarchy, two nodes are connected by a directed edge, where the source node is the child node and the target node is the parent node. All edges are labeled either as *subClassOf* or *equivalentClass*.

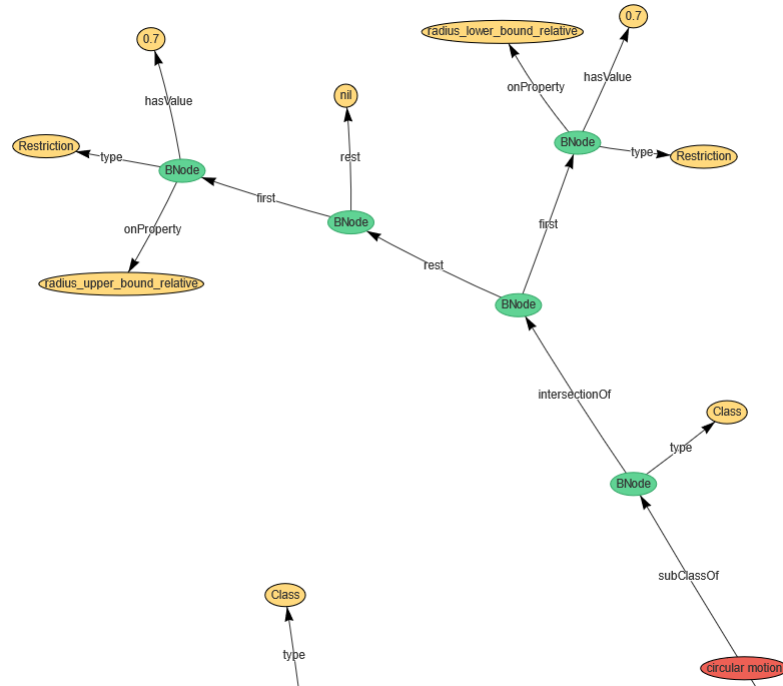


Figure 7.4: Visualize parsed class restrictions

To achieve a compact representation of class expressions, we recursively traverse the parsed class expression, searching for essential information such as the relations used and their corresponding values. All blank nodes are implicitly skipped and not visualized. Additionally, meta information are excluded.

The visualizer can also handle nested expressions, as well as intersections and unions. Each **AND** node indicates that all connected elements belong to the intersection. Similarly, each **OR** node indicates that all connected elements belong to the union.

In the figure 7.5, the following expression is visualized:

CircularMotion *subClassOf* (radius upper bound relative value 0.7) and (radius lower bound relative value 0.7)

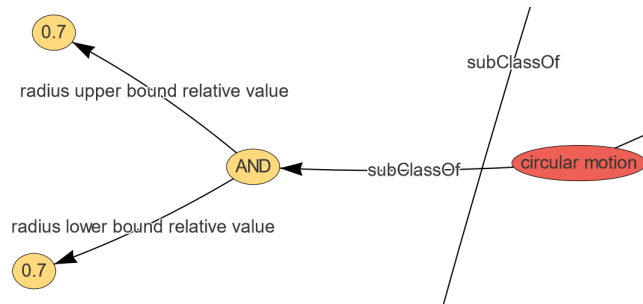


Figure 7.5: Compact representation of class expression

7.2.3 Load Ontology

By pressing the *Load Ontology* button, users can load a wide range of ontologies that have the *.owl* file extension. Upon selecting the ontology, the *Knowledge Graph Visualization Tool* begins parsing the ontology to process the ontology. Once the processing is complete, a visual representation of the ontology is created, which can be explored by the users freely.

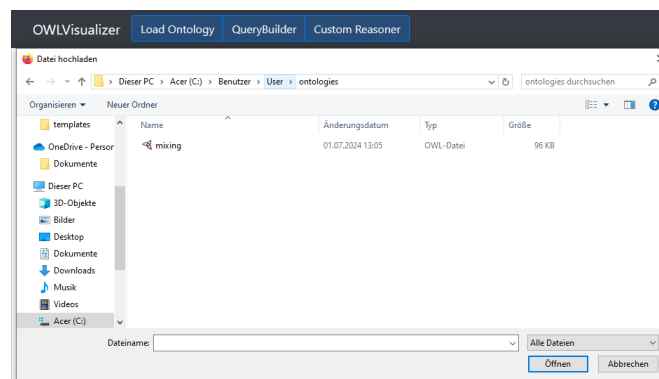
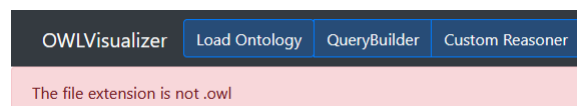


Figure 7.6: Loading Ontology

Attempting to load any other file types results in an error message being displayed on the website. When a user tries to upload a file that does not have the *.owl* extension, the *Knowledge Graph Visualization Tool* notifies the user that the file extension is not supported by this tool, via an error message.

Figure 7.7: **Error:** Unsupported File Type

Additionally, if an ontology can't be parsed, an error message is thrown that ontology couldn't be loaded properly. Whenever this issue is encountered during the parsing of the selected ontology, it immediately stops and triggers an error notification. This message informs the user that there was a problem with loading the ontology, possibly due to a formatting error or an unsupported structure within the file. This error message informs the user of this particular issue; the *Knowledge Graph Visualization Tool* doesn't attempt to deal with this problem.

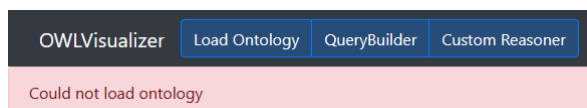


Figure 7.8: **Error:** Ontology Load Failure

7.2.4 Search Classes

The *Knowledge Graph Visualization Tool* includes a feature that suggests classes, which are visualized as nodes, for users to focus on, improving their ability to navigate and understand complex networks.

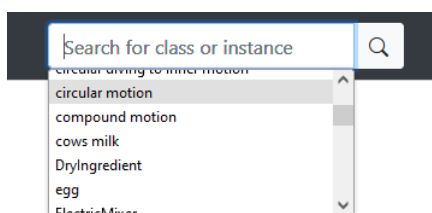


Figure 7.9: Search bar suggesting available classes

The key implementation for this feature is a search bar that suggests all visualized classes. Any nodes belonging to a class restriction are not suggested. The available suggestions are sorted in descending order, allowing users to easily search for classes and explore the class hierarchy without assuming prior familiarity with the ontology.

generated, which the robot can execute to successfully complete its task. To achieve this, the form data is processed, and a reasoning process is initiated to run inference over our *SWRL* rules, extract all inferred knowledge, and create a visualization and a sequence of actions.



Figure 7.12: Visualize Graph and Sequence of Actions.

7.2.6 Query Builder

The *Query Builder* is a feature that provides users with suggested triples, allowing them to explore the graph in an intuitive manner. No *SPARQL* knowledge is required. This feature generates a visualization of the selected triples and constructs a *SPARQL* query, which can be modified to the users needs.

Triple Matching Triple Matching allows users to select a subject, predicate, and object to explore the graph. At the beginning of the triple matching process, all classes are available as subjects.

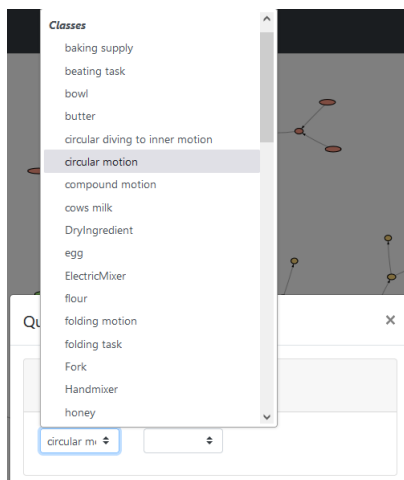


Figure 7.13: View Possible Subjects for Selection

Only predicates that appear as edge labels connected to the subject in the graph are suggested.

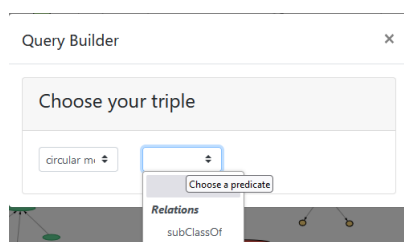


Figure 7.14: View Possible Predicates for Selection

Similarly, only objects connected to the subject via the selected edge are proposed.

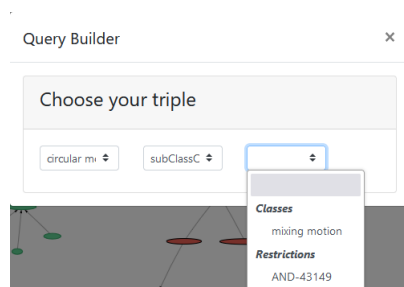


Figure 7.15: View Possible Objects for Selection

Once a triple is selected, the user can continue to explore the graph based on the previously chosen triples.

All complex class expressions are processed in a single triple matching step, whereas navigating through the class hierarchy occurs incrementally.

Whenever a triple is selected, the *Query Builder* creates a tab with two items, one item visualizes the graph, the other item contains a *SPARQL* query.

View Graph In the *View Graph*, users can inspect the graph based on their selection of triples. This view is a subgraph, which makes it easier to inspect the relationships between classes and classes with complex class expressions. Each class expression is displayed in a separate view, allowing users to focus on each class expression independently, which also reduces clutter in the graph. The class hierarchy has its own view, which can be incrementally updated with new classes to view hierarchical relationships. Separating the views ensures that users can navigate and explore different aspects of the graph, while also improving their comprehension of the ontology.

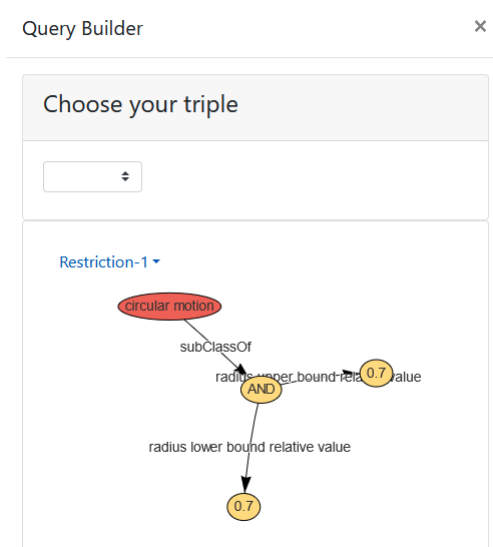


Figure 7.16: Graph View

SPARQL Query A *SPARQL ASK* query is generated for each chosen triple. If the triple is querying the hierarchy, the *SPARQL* query for querying the hierarchy is updated with the newest triples. For each complex class expression, a separate query corresponds to the view of the subgraph.

This query is a queryable triple pattern that asks if this query pattern has a solution or not. Users can then independently modify and adapt this pattern to suit their specific

needs. This feature will help users unfamiliar with querying ontologies by providing them with the triple pattern.

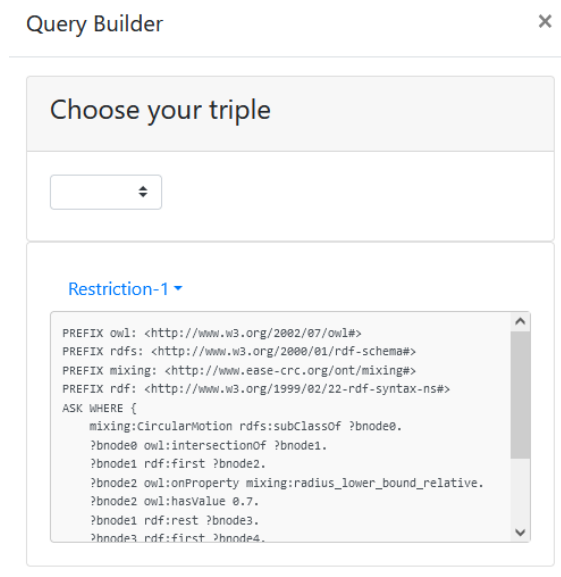


Figure 7.17: *SPARQL View*

7.3 Limitations

Using *vis.js* and Visualization of NamedIndividuals The graph visualization library *vis.js* suffers from poor performance when handling large graphs, acknowledged by its developers [36]. This limitation primarily stems from the physics simulation used in its force-directed graph layout, which attempts to arrange nodes by separating and clustering them based on their connections. As the number of interconnected nodes and edges grows, this process becomes increasingly time-consuming, leading to decreased performance. While capable of displaying relationships and attributes of instance data effectively, *vis.js* is not optimized for handling large ontologies with numerous instantiated classes. Due to these limitations, functionality for visualizing such ontologies has not been implemented.

Inference and Reasoning Although we have developed a custom reasoner tailored specifically for the *mixing* ontology, the current implementation does not support reasoning over other ontologies apart from *mixing*. This decision was made due to runtime concerns, particularly regarding the computational complexity of *SWRL* rules. Loading an ABox ontology, characterized by numerous NamedIndividuals and a complex class hierarchy, into this framework could severely impact performance. Such intensive computations could potentially disrupt the usability of other features within the *Knowledge Graph Visualization Tool*.

8 Conclusion

8.1 Summary

This master’s thesis serves as a proof of concept to demonstrate how a robotic system like the *PR2* [1], utilizing the cognitive architecture *CRAM* [8], can make knowledge-based decisions and perform mixing actions in a simulation.

In order to implement the motions in *CRAM* [8], one must first understand the relationship of the motions to the overall context of the task the robot needs to solve. What we have discovered, primarily conducted through video analyses of various cooking videos, is that the motions are primarily dependent on the ingredients in the context of mixing task, which variations we highlighted in our work. The containers and tools used do not directly influence the motion itself, but they do modify the action range of the motions (see [Data Acquisition](#)).

The model we use—a knowledge base (see [Data Representation](#))—captures various important aspects of a mixing action. It includes information about the ingredients being mixed, the types of motions that can be performed, the type of container used for storing and mixing ingredients, and the tools needed for mixing. By keeping the model simple, we ensure that the robot can infer motions using the rule-based language *SWRL* (3.2.2).

Our modeled motions do not specify how a motion is executed but provide capabilities via parameters to adapt to different containers and tools. The implementation of each motion—**Circular**, **Folding**, **Horizontal Elliptical**, and **Whirlstorm**—is available in *PyCram* [2], and any robotic system that can use *CRAM* [8] can perform these motions.

We also transitioned from theory to practice and implemented and tested the *Mixing Action Designator*, which actively uses the *mixing* ontology. Through evaluation, we tested as many scenarios as possible to demonstrate how the ontology infers different motions, which the robot executed (see [Execution on the Robot](#)).

In addition to our proof of concept, we developed a different way to visualize ontologies by focusing on complex class expressions, which hasn’t been done before (see [Knowledge Graph Visualization Tool](#)). With the *Knowledge Graph Visualization Tool*, a user can query complex class expressions without needing to know how to write *SPARQL* queries. The *Knowledge Graph Visualization Tool* generates a *SPARQL* query for the user, using triple matching to get the desired graph. This query can be customized by changing its

contents without altering the triple matching. Another feature of the *Knowledge Graph Visualization Tool* is the *Custom Inference*, which reasons over the *mixing* ontology, to infer motions from task and ingredients. As a result a visualized graph and a sequence of actions are generated, to highlight the reasoning.

8.2 Future Work

This proof of concept was developed in *CRAM* [8] in simulation. Simulations simplify many aspects: the robot has ground truth for each available object, the ingredients to mix, the tools for mixing, and the container to mix in. To advance our work, this proof of concept should be implemented on a real robot to assess the increased challenges in real-world scenarios.

Regarding future work, several limitations emerged during the implementation of our proof of concept. Our motions are essentially 2D, with one axis remaining unchanged throughout the motion. Transitioning from simulation to the real world will require incorporating depth into each motion to ensure the substances are properly mixed.

Another limiting aspect is the lack of collision detection. Currently, we ensure that the tool does not collide with objects concerning only 2 out of 3 axes. If the tool is placed too low and the container has a spherical rather than a cuboid shape, the tool will eventually collide with the object. To address this limitation, we need depth-based perception and various techniques to determine the optimal height for the tool during the execution of a mixing action.

Once we transition to the real world, we lose the ground truth on all relevant objects needed for mixing. We will need perception frameworks capable of detecting and locating objects in a scene, picking and placing objects, and performing various actions, such as pouring, to fill a container with ingredients.

In simulation, the robot will not fail unless the inference fails. As long as a motion is inferred, the mixing will be executed. This approach needs to be adjusted for real-world application.

One last important aspect is the implementation of failure handling, which is currently completely absent. Failure handling should address various aspects of robot actions, such as scenarios where the robot plan cannot proceed due to reasons like incorrect perception or grasping of an object. Countermeasures should be developed for such potential scenarios to still allow plan execution.

Bibliography

- [1] *Pr2*. <https://robotsguide.com/robots/pr2> [Accessed: 03.05.2024].
- [2] *Pycram*. <https://github.com/cram2/pycram> [Accessed: 03.05.2024].
- [3] *wikihow*. <https://www.wikihow.com/Main-Page> [Accessed: 03.05.2024].
- [4] *Wikihow instruction analysis for robot manipulation*. <https://github.com/Food-Ninja/WikiHow-Instruction-Extraction>.
- [5] Bastian, Mathieu, Sebastien Heymann, and Mathieu Jacomy: *Gephi: The open graph viz platform*, 2009. <https://gephi.org/>, Accessed: 2024-05-15.
- [6] Beetz, Michael, Philipp Cimiano, Michaela Kümpel, Enrico Motta, Ilaria Tiddi, and Jan Philipp Töberg: *Translating actionable knowledge graphs into robot action execution*. In *HHAI-WS 2024: Workshops at the Third International Conference on Hybrid Human-Artificial Intelligence (HHAI)*, June 10-11, Malmö, Sweden, 2024. in press.
- [7] Beetz, Michael, Ulrich Klank, Ingo Kresse, Alexis Maldonado, Lorenz Mösenlechner, Dejan Pangercic, Thomas Rühr, and Moritz Tenorth: *Robotic roommates making pancakes*. pages 529–536, October 2011.
- [8] Beetz, Michael, Lorenz Mösenlechner, and Moritz Tenorth: *CRAM – A Cognitive Robot Abstract Machine for Everyday Manipulation in Human Environments*. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1012–1017, Taipei, Taiwan, October 18-22 2010.
- [9] Berkely, University of California: *Framenet*. <https://framenet.icsi.berkeley.edu/> [Accessed: 03.05.2024].
- [10] Bekler, Daniel, Robert Porzel, Mihai Pomarlan, Abhijit Vyas, Sebastian Höffner, Michael Beetz, Rainer Malaka, and John Bateman: *Foundations of the socio-physical model of activities (soma) for autonomous robotic agents*. April 2021.
- [11] Bollini, Mario, Jennifer K Barry, and Daniela Rus: *Bakebot: Baking cookies with the pr2*. 2011. <https://api.semanticscholar.org/CorpusID:18098545>.

- [12] Coumans, Erwin and Yunfei Bai: *Pybullet, a python module for physics simulation for games, robotics and machine learning*. <http://pybullet.org>, 2016–2021.
- [13] Danno, Dylan, Simon Hauser, and Fumiya Iida: *Robotic cooking through pose extraction from human natural cooking using openpose*. In Ang Jr, Marcelo H., Hajime Asama, Wei Lin, and Shaohui Foong (editors): *Intelligent Autonomous Systems 16*, pages 288–298, Cham, 2022. Springer International Publishing, ISBN 978-3-030-95892-3.
- [14] Dictionary, Oxford English: *mix verb, meaning and use*, 2002. https://www.oed.com/dictionary/mix_v?tab=meaning_and_use [Accessed: 03.05.2024].
- [15] Dooley, D. M., E. J. Griffiths, G. S. Gosal, P. L. Buttigieg, R. Hoehndorf, M. C. Lange, L. M. Schriml, F. S. L. Brinkman, and W. W. L. Hsiao: *Foodon: a harmonized food ontology to increase global food traceability, quality control and data integration*. NPJ Science of Food, 2:23, Dec 2018.
- [16] Grinberg, Miguel: *Flask web development: developing web applications with python*. "O'Reilly Media, Inc.", 2018.
- [17] Hassouna, Vanessa, Alina Hawkin, and Michael Beetz: *Steps towards generalized manipulation action plans - tackling mixing task*. In *2024 Actionable Knowledge Representation and Reasoning for Robots (AKR³) Workshop*, 2024. Accepted.
- [18] Horrocks, Ian, Peter F. Patel-Schneider, Harold Boley, Said Tabet, Benjamin Grosz, and Mike Dean: *SWRL: A semantic web rule language combining OWL and RuleML*. W3C Member Submission, May 2004. <http://www.w3.org/Submission/SWRL/>, Last access on Dez 2008 at: <http://www.w3.org/Submission/SWRL/>.
- [19] Krech, Daniel, Gunnar Aastrand Grimnes, Graham Higgins, Jörn Hees, Iwan Aucamp, Niklas Lindström, Natanael Arndt, Ashley Sommer, Edmond Chuc, Ivan Herman, Alex Nelson, Jamie McCusker, Tom Gillespie, Thomas Kluyver, Florian Ludwig, Pierre Antoine Champin, Mark Watts, Urs Holzer, Ed Summers, Whit Morriss, Donny Winston, Drew Perttula, Filip Kovacevic, Remi Chateaneu, Harold Solbrig, Benjamin Cogrel, and Veyndan Stuart: *RDFLib*, August 2023. <https://github.com/RDFLib/rdfLib>.
- [20] Kuempel, Michaela and Vanessa Hassouna: *Using web knowledge for flexible meal preparation*. <https://github.com/Food-Ninja/WebKat-MealRobot>, 2024.
- [21] Kümpel, Michaela: *Actionable Knowledge Graphs - how daily activity applications can benefit from embodied web knowledge*. PhD thesis, 2024.
- [22] Kümpel, Michaela, Jan Phillip Töberg, Vanessa Hassouna, Phillip Cimiano, and Michael Beetz: *Towards a knowledge engineering methodology for flexible robot manipulation in everyday tasks*. In *Actionable Knowledge Representation and Reasoning for Robots (AKR³) Workshop*, 2024. Accepted.

- soning for Robots (AKR3) at European Semantic Web Conference (ESWC), 2024. <https://ceur-ws.org/Vol-3749/akr3-04.pdf>.
- [23] Kümpel, Michaela, Abhijit Vyas, Vanessa Hassouna, and Michael Beetz: *Task learning using actionable knowledge graphs*. In *ICRA40- 40th Anniversary of the IEEE International Conference on Robotics and Automation*, 2024. in press.
- [24] Lamy, Jean Baptiste: *Owlready: Ontology-oriented programming in python with automatic classification and high level constructs for biomedical ontologies*. Artificial Intelligence in Medicine, 80:11–28, 2017, ISSN 0933-3657. <https://www.sciencedirect.com/science/article/pii/S0933365717300271>.
- [25] Neo4j, Inc.: *Neo4j graph platform*, 2024. <https://neo4j.com/>, Accessed: 2024-05-15.
- [26] Ohene, Michael: *A proposed general formula to create and analyze baking recipes*. 2017.
- [27] ontotext: *What are ontologies?* <https://www.ontotext.com/knowledgehub/fundamentals/what-are-ontologies/>, visited on 2024-06-06.
- [28] Oxford Learner’s Dictionaries: *Crockery*, 2024. <https://www.oxfordlearnersdictionaries.com/definition/english/crockery>, Accessed: 2024-06-12.
- [29] Oxford Learner’s Dictionaries: *Hyponym*, 2024. https://www.oxfordlearnersdictionaries.com/definition/american_english/hyponym, Accessed: 2024-06-05.
- [30] Shearer, Rob, Boris Motik, and Ian Horrocks: *Hermit: A highly-efficient owl reasoner*. In *OWLED*, volume 432, pages 91–108, 2008.
- [31] Sirin, Evren, Bijan Parsia, Bernardo Cuenca Grau, Aditya Kalyanpur, and Yarden Katz: *Pellet: A practical owl-dl reasoner*. In *Web Semantics: Science, Services and Agents on the World Wide Web*, volume 5, pages 51–53. Elsevier, 2007.
- [32] Stanford Artificial Intelligence Laboratory et al.: *Robotic operating system*. <https://www.ros.org>.
- [33] Stardog Union: *Stardog Explorer*, 2024. <https://www.stardog.com/products/explorer/>, Accessed: 2024-05-15.
- [34] Tina, Van: *Exploring the use of designators for improved contextualization in robotic mixing tasks*, 2023.
- [35] University, Princeton: *Wordnet, a lexical database for english*. <https://wordnet.princeton.edu/> [Accessed: 03.05.2024].

-
- [36] Vis.js: *vis.js - a dynamic, browser based visualization library*. <https://visjs.org/>. Accessed: 2024-06-25.
- [37] Wikipedia contributors: *Levenshtein distance*, 2024. https://en.wikipedia.org/wiki/Levenshtein_distance, Accessed: 2024-07-07.
- [38] Xian, Zhou, Bo Zhu, Zhenjia Xu, Hsiao Yu Tung, Antonio Torralba, Katerina Fragkiadaki, and Chuang Gan: *Fluidlab: A differentiable environment for benchmarking complex fluid manipulation*, 2023.

9 Appendix

9.1 Video Analysis

In the section [Task Analysis and Defintion](#), a simplified table was presented, which is intended to represent a video analysis. From this analysis, we extract important information that is used for modeling the knowledge base. In the appendix, we present more video analyses and provide the corresponding sources.

Note:

- We provide each table entry with a number, and the corresponding sources are mentioned after the table.

Nr.	Task	Ingredients	Description
1	Mixing	Eggs, melted butter	Circular, from the inside to the outside.
2	Mixing	dry yeast, water	Whirlstorm-like motion.
3	Mixing	dry yeast, water, flour, salt	Whirlstorm-like motion.
4	Mixing	milk, egg, flour	Whirlstorm-like motion.
5	Whisking	milk, egg, flour, salt	Whirlstorm-like motion.
6	Stirring	Semi-Liquid batter, milk	Whirlstorm-like motion.
7	Whisking	Powder ingredients	Whirlstorm-like motion.
8	Beating	Semi-Liquid ingredients, Powder ingredients	Horizontal Eliptical motion
9	Mixing	milk, Powder ingredients	Whirlstorm-like motion.
10	Whisking	flour, salt, baking soda	Whirlstorm-like motion.
11	Beating	egg yolk	Horizontal Eliptical motion.
12	Stirring	egg yolk, parmesan, pepper	Horizontal Eliptical and Whirlstorm motion.
13	Mixing	brown sugar, butter	Whirlstorm-like motion.

Table 9.1: Video analysis

14	Stirring	eggs, smashed bananas, Powder ingredients	Horizontal Elliptical and Whirlstorm motion.
15	Beating	egg yolk, sugar	Horizontal Elliptical motion.
16	Stirring	Liquid mixture, vanilla extract	Circular motion.
17	Beating	egg white	Horizontal Elliptical motion.
18	Folding	Semi-Liquid mixture	Folding motion.
19	Whisking	Liquid mixture	Whirlstorm-like motion.
20	Whisking	Semi-Liquid mixture	Whirlstorm-like motion.
21	Folding	Semi-Liquid, powder ingredients	Folding motion.
22	Stirring	sugar, liquid	Whirlstorm-like motion.
23	Beating	butter, sugar	Horizontal Elliptical motion.
24	Beating	Semi-Liquid mixture, liquid	Horizontal Elliptical motion and Whirlstorm-like motion.
25	Mixing	butter, cream cheese	Whirlstorm and Horizontal Elliptical Motion.

Table 9.2: Video analysis

Source

- 1: <https://www.youtube.com/watch?v=s9r-CxnCXkg>
- 2,3: <https://www.youtube.com/watch?v=1-SJGQ2HLP8>
- 4: <https://www.wikihow.com/Make-Pancakes>
- 5,6: <https://www.youtube.com/watch?v=iwxJTIXIoFo>
- 7,8,9: <https://www.youtube.com/watch?v=vkcHmpKxFwg>
- 10: <https://www.youtube.com/watch?v=loqCY9b7aec>
- 11,12: <https://www.youtube.com/watch?v=3AAdKl1UYZs>
- 13,14: <https://www.wikihow.com/Make-Banana-Bread>
- 15-20: <https://www.wikihow.life/Make-a-Sponge-Cake>
- 21: <https://www.wikihow.com/Make-an-Easy-Sponge-Cake>
- 22-25: <https://www.wikihow.com/Make-Cake-Icing>

9.2 Evaluation

In the appendix, we would like to build on the concept presented in [Evaluation](#). Here we will present the tables in more detail and cover as many scenarios as possible. As a reminder: A combination of a task and a set of ingredients infers motions, which are in turn associated with parameters. These parameters are used to adjust the motion depending on the chosen tool and container. Ultimately, it must be calculated in which range the motion can be executed. The calculation is as follows:

$$\begin{aligned} \text{radius_upper_bound} &= ((\text{dim}[0] * \text{radius_bounds}[0]) - \\ &\quad \max(\text{dim2}[0], \text{dim2}[1])) / 2 \\ \\ \text{radius_lower_bound} &= \max(0, ((\text{dim}[0] * \text{radius_bounds}[1]) - \\ &\quad \max(\text{dim2}[0], \text{dim2}[1])) / 2) \end{aligned}$$

where the dimensional parameters are defined as follows:

$$\begin{aligned} \text{dim} &= [\max(\text{obj_dim}[0], \text{obj_dim}[1]), \\ &\quad \min(\text{obj_dim}[0], \text{obj_dim}[1]), \text{obj_dim}[2]] \\ \\ \text{dim2} &= [\max(\text{tool_dim}[0], \text{tool_dim}[1]), \\ &\quad \min(\text{tool_dim}[0], \text{tool_dim}[1]), \text{tool_dim}[2]] \end{aligned}$$

tool_dim and *obj_dim* correspond to the dimensions of the tool and the container, respectively. With this calculation, it is then possible to define an action radius to perform the motion safely, meaning without the tool, for example, hitting the edge of the container.

In the following, we would like to prove that our system adapts to the environment and calculates the action radii depending on the given dimensions.

Notes:

- The radius bounds are a list of two elements where the first element corresponds to the radius of the upper bound and the second element corresponds to the radius of the lower bound.
- Some combinations of task and ingredient result in the same motion and are therefore summarized in this case.
- It can happen that the action radius is negative; in this case, fine-tuning is necessary. Generally, a motion should not be performed with a negative action radius. However, for relatively small radii, an exception could be made.
- The tools spoon and fork are not considered separately as they are very similar in relevant form and size.

- The folding task infers the same parameters regardless of the ingredients.
- Every value for the radii is in meters.
- **Whisk Dimensions:** $0.11, 0.08, 0.31$
- **Wooden Spoon Dimensions:** $0.09, 0.095, 0.29$
- **Fork Dimensions:** $0.06, 0.04, 0.25$
- **Salad Bowl Dimensions:** $0.25, 0.25, 0.11$
- **Pot Dimensions:** $0.31, 0.37, 0.11$
- **Small Bowl Dimensions:** $0.14, 0.14, 0.07$

9.2.1 Mixing Task

Liquid, Powder, Liquid and Powder, Liquid and Semi-Liquid

- **Inferred Motions:** *Whirlstorm Motion*
- **Inferred Parmaters:** $0.7, 0.0$

Tool	Container	Action Radius
Whisk	Salad Bowl	$[0.033, 0]$
Whisk	Pot	$[0.075, 0]$
Whisk	Small Bowl	$[-0.004, 0]$
Wooden Spoon	Salad Bowl	$[0.04, 0]$
Wooden Spoon	Pot	$[0.082, 0]$
Wooden Spoon	Small Bowl	$[0.003, 0]$
Fork	Salad Bowl	$[0.058, 0]$
Fork	Pot	$[0.1, 0]$
Fork	Small Bowl	$[0.03, 0]$

Table 9.3: Mixing Task which infer Whirlstorm Motion

Semi-Liquid, Powder and Semi-Liquid

- **Inferred Motions:** *Whirlstorm Motion, Horizontal Eliptical Motion*
- **Inferred Parmaters:** $0.7, 0.0$ and *ellipse shift:* 0.04

Tool	Container	Action Radius
Whisk	Salad Bowl	[0.033, 0]
Whisk	Pot	[0.075, 0]
Whisk	Small Bowl	[-0.004, 0]
Wooden Spoon	Salad Bowl	[0.04, 0]
Wooden Spoon	Pot	[0.082, 0]
Wooden Spoon	Small Bowl	[0.003, 0]
Fork	Salad Bowl	[0.058, 0]
Fork	Pot	[0.1, 0]
Fork	Small Bowl	[0.03, 0]

Table 9.4: Mixing Task that infer Whirlstorm and Horizontal Eliptical Motion

9.2.2 Stirring Task

Liquid, Liquid and Powder

- **Inferred Motion:** *Circular Motion*
- **Inferred Parameters:** *0.7, 0.7*

Tool	Container	Action Radius
Whisk	Salad Bowl	[0.033, 0.033]
Whisk	Pot	[0.075, 0.075]
Whisk	Small Bowl	[-0.004, -0.004]
Wooden Spoon	Salad Bowl	[0.04, 0.04]
Wooden Spoon	Pot	[0.082, 0.082]
Wooden Spoon	Small Bowl	[0.003, 0.003]
Fork	Salad Bowl	[0.058, 0.058]
Fork	Pot	[0.1, 0.1]
Fork	Small Bowl	[0.03, 0.03]

Table 9.5: Stirring Task which infer Circular Motion

Powder

- **Inferred Motions:** *Whirlstorm Motion*
- **Inferred Parmaters:** *0.7, 0.0*

Tool	Container	Action Radius
Whisk	Salad Bowl	[0.033, 0]
Whisk	Pot	[0.075, 0]
Whisk	Small Bowl	[-0.004, 0]
Wooden Spoon	Salad Bowl	[0.04, 0]
Wooden Spoon	Pot	[0.082, 0]
Wooden Spoon	Small Bowl	[0.003, 0]
Fork	Salad Bowl	[0.058, 0]
Fork	Pot	[0.1, 0]
Fork	Small Bowl	[0.03, 0]

Table 9.6: Stirring Task which infer Whirlstorm Motion

Semi-Liquid, Semi-Liquid and Liquid, Semi-Liquid and Powder

- **Inferred Motions:** *Whirlstorm Motion, Horizontal Eliptical Motion*
- **Inferred Parmaters:** *0.7, 0.0 and ellipse shift: 0.04*

Tool	Container	Action Radius
Whisk	Salad Bowl	[0.033, 0]
Whisk	Pot	[0.075, 0]
Whisk	Small Bowl	[-0.004, 0]
Wooden Spoon	Salad Bowl	[0.04, 0]
Wooden Spoon	Pot	[0.082, 0]
Wooden Spoon	Small Bowl	[0.003, 0]
Fork	Salad Bowl	[0.058, 0]
Fork	Pot	[0.1, 0]
Fork	Small Bowl	[0.03, 0]

Table 9.7: Stirring Task that infer Whirlstorm and Horizontal Eliptical Motion

9.2.3 Beating Task

Liquid, Liquid and Powder

- **Inferred Motions:** *Whirlstorm Motion*
- **Inferred Parmaters:** *0.7, 0.0*

Tool	Container	Action Radius
Whisk	Salad Bowl	[0.033, 0]
Whisk	Pot	[0.075, 0]
Whisk	Small Bowl	[-0.004, 0]
Wooden Spoon	Salad Bowl	[0.04, 0]
Wooden Spoon	Pot	[0.082, 0]
Wooden Spoon	Small Bowl	[0.003, 0]
Fork	Salad Bowl	[0.058, 0]
Fork	Pot	[0.1, 0]
Fork	Small Bowl	[0.03, 0]

Table 9.8: Beating Task which infer Whirlstorm Motion

Powder, Semi-Liquid, Liquid and Semi-Liquid

- **Inferred Motions:** *Whirlstorm Motion, Horizontal Eliptical Motion*
- **Inferred Parmaters:** *0.7, 0.0 and ellipse shift: 0.04*

Tool	Container	Action Radius
Whisk	Salad Bowl	[0.033, 0]
Whisk	Pot	[0.075, 0]
Whisk	Small Bowl	[-0.004, 0]
Wooden Spoon	Salad Bowl	[0.04, 0]
Wooden Spoon	Pot	[0.082, 0]
Wooden Spoon	Small Bowl	[0.003, 0]
Fork	Salad Bowl	[0.058, 0]
Fork	Pot	[0.1, 0]
Fork	Small Bowl	[0.03, 0]

Table 9.9: Beating Task which infer Whirlstorm and Horizontal Eliptical Motion

Powder and Semi-Liquid

- **Inferred Motions:** *Horizontal Eliptical Motion*
- **Inferred Parmaters:** *0.7, 0.0 and ellipse shift: 0.04*

Tool	Container	Action Radius
Whisk	Salad Bowl	[0.033, 0]
Whisk	Pot	[0.075, 0]
Whisk	Small Bowl	[-0.004, 0]
Wooden Spoon	Salad Bowl	[0.04, 0]
Wooden Spoon	Pot	[0.082, 0]
Wooden Spoon	Small Bowl	[0.003, 0]
Fork	Salad Bowl	[0.058, 0]
Fork	Pot	[0.1, 0]
Fork	Small Bowl	[0.03, 0]

Table 9.10: Beating Task which infer Horizontal Elliptical Motion

9.2.4 Whisking Task

Liquid, Powder, Liquid and Powder, Liquid and Semi-Liquid, Semi-Liquid and Powder

- **Inferred Motions:** *Whirlstorm Motion*
- **Inferred Parmaters:** *0.7, 0.0*

Tool	Container	Action Radius
Whisk	Salad Bowl	[0.033, 0]
Whisk	Pot	[0.075, 0]
Whisk	Small Bowl	[-0.004, 0]
Wooden Spoon	Salad Bowl	[0.04, 0]
Wooden Spoon	Pot	[0.082, 0]
Wooden Spoon	Small Bowl	[0.003, 0]
Fork	Salad Bowl	[0.058, 0]
Fork	Pot	[0.1, 0]
Fork	Small Bowl	[0.03, 0]

Table 9.11: Whisking Task which infer Whirlstorm Motion

Semi-Liquid

- **Inferred Motions:** *Horizontal Eliptical Motion*
- **Inferred Parmaters:** *0.7, 0.0* and *ellipse shift: 0.04*

Tool	Container	Action Radius
Whisk	Salad Bowl	[0.033, 0]
Whisk	Pot	[0.075, 0]
Whisk	Small Bowl	[-0.004, 0]
Wooden Spoon	Salad Bowl	[0.04, 0]
Wooden Spoon	Pot	[0.082, 0]
Wooden Spoon	Small Bowl	[0.003, 0]
Fork	Salad Bowl	[0.058, 0]
Fork	Pot	[0.1, 0]
Fork	Small Bowl	[0.03, 0]

Table 9.12: Whisking Task which infer Horizontal Eliptical Motion

9.2.5 Folding Task

Every Ingredient

- **Inferred Motions:** *Folding Motion*
- **Inferred Parmaters:** *0.7, 0.0; repetitive folding rotation shift: 22.5 and folding rotation shift: 90*

Tool	Container	Action Radius
Whisk	Salad Bowl	[0.033, 0]
Whisk	Pot	[0.075, 0]
Whisk	Small Bowl	[-0.004, 0]
Wooden Spoon	Salad Bowl	[0.04, 0]
Wooden Spoon	Pot	[0.082, 0]
Wooden Spoon	Small Bowl	[0.003, 0]
Fork	Salad Bowl	[0.058, 0]
Fork	Pot	[0.1, 0]
Fork	Small Bowl	[0.03, 0]

Table 9.13: Folding Task